

Design Patterns

Structural Patterns

Adapter

Akın Kaldıroğlu

backend
guru

Problem



Problem - I

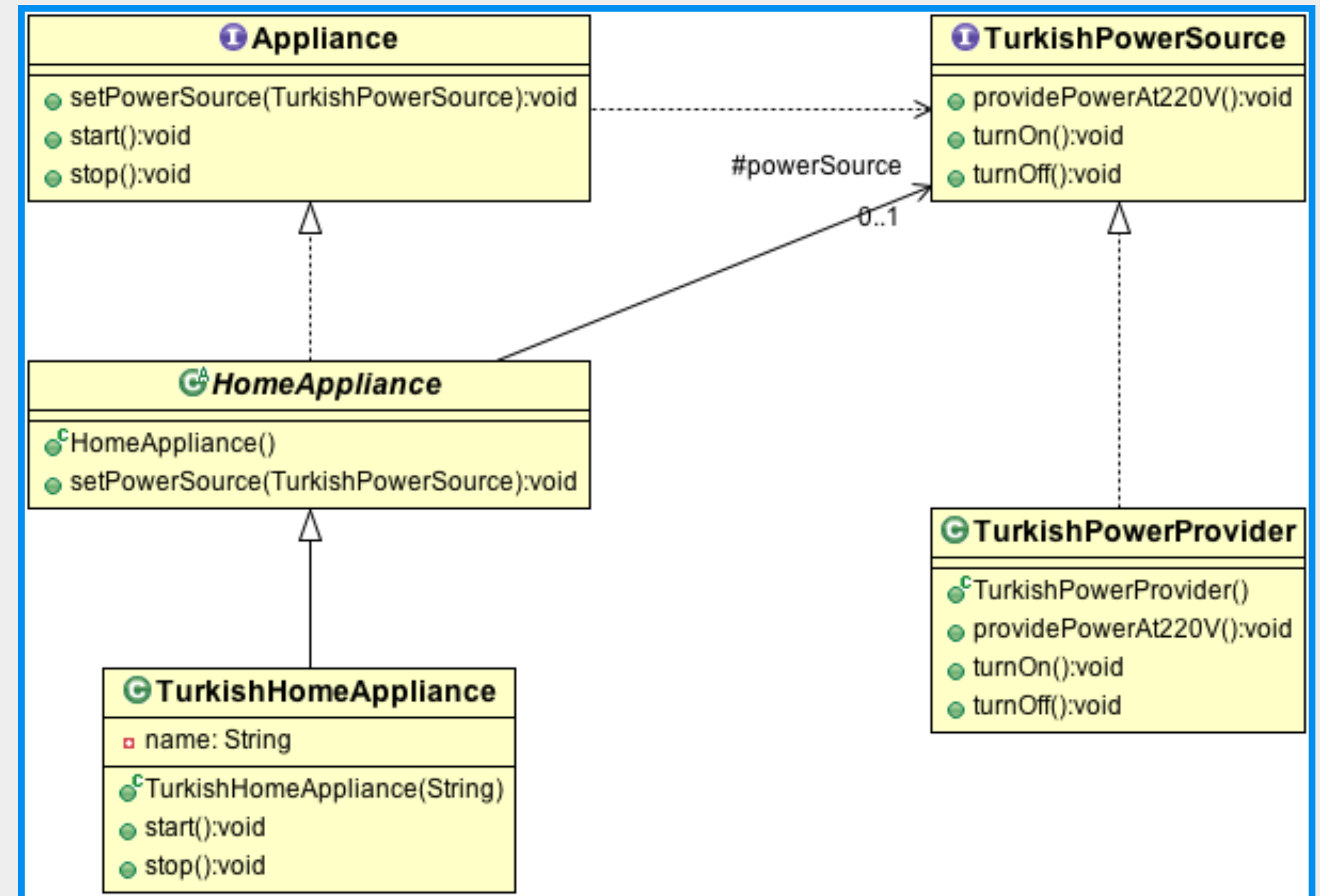


- We often want to use structures with different interfaces together.
- This can lead to increasingly complex clients.
- Clients often need to communicate with multiple interfaces and perform the necessary conversions between them.
- This is a problematic approach.

electricity



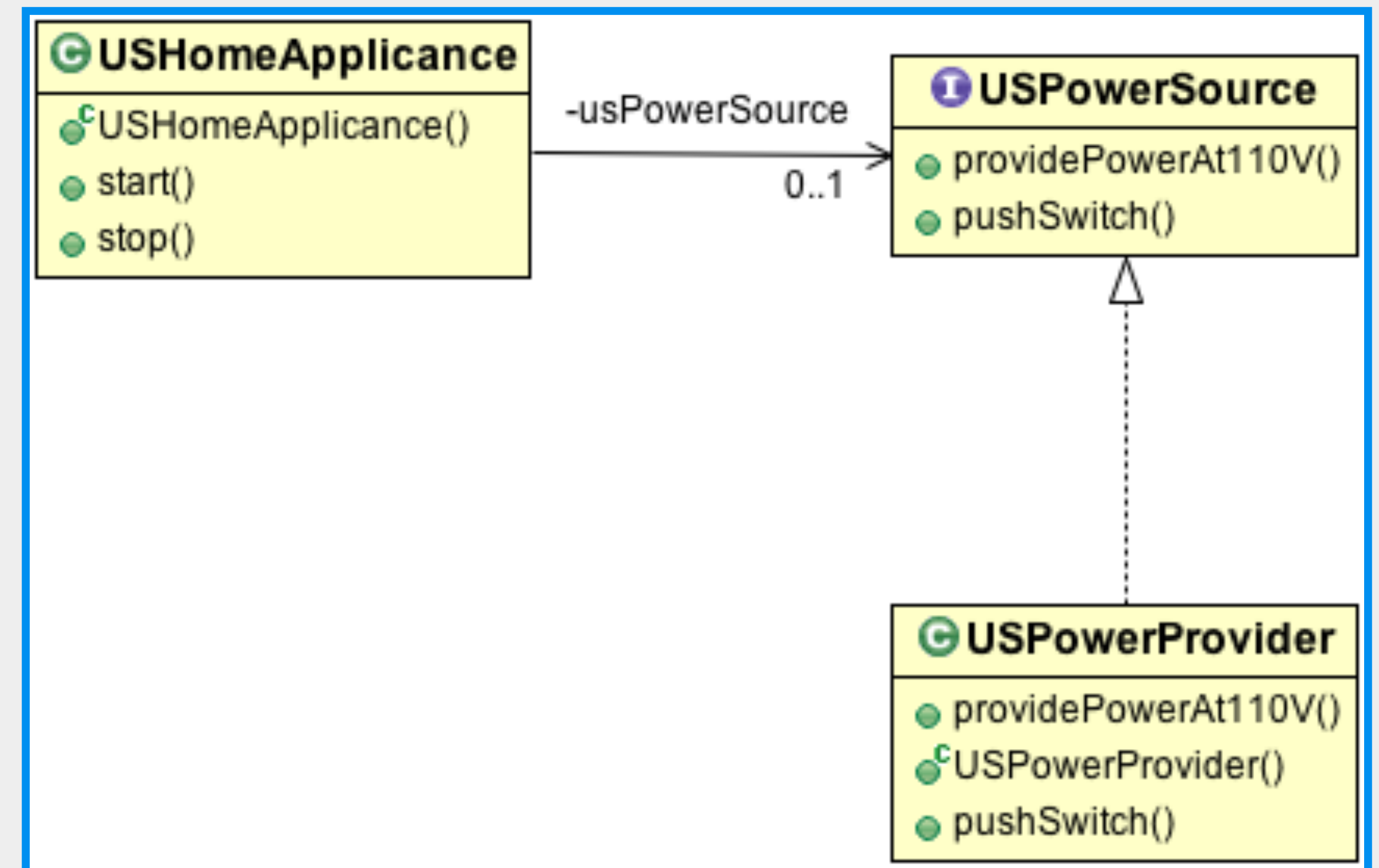
- `electricity.domain`
- Consider the electrical appliances in use in Turkey and the infrastructure that powers them.



electricity

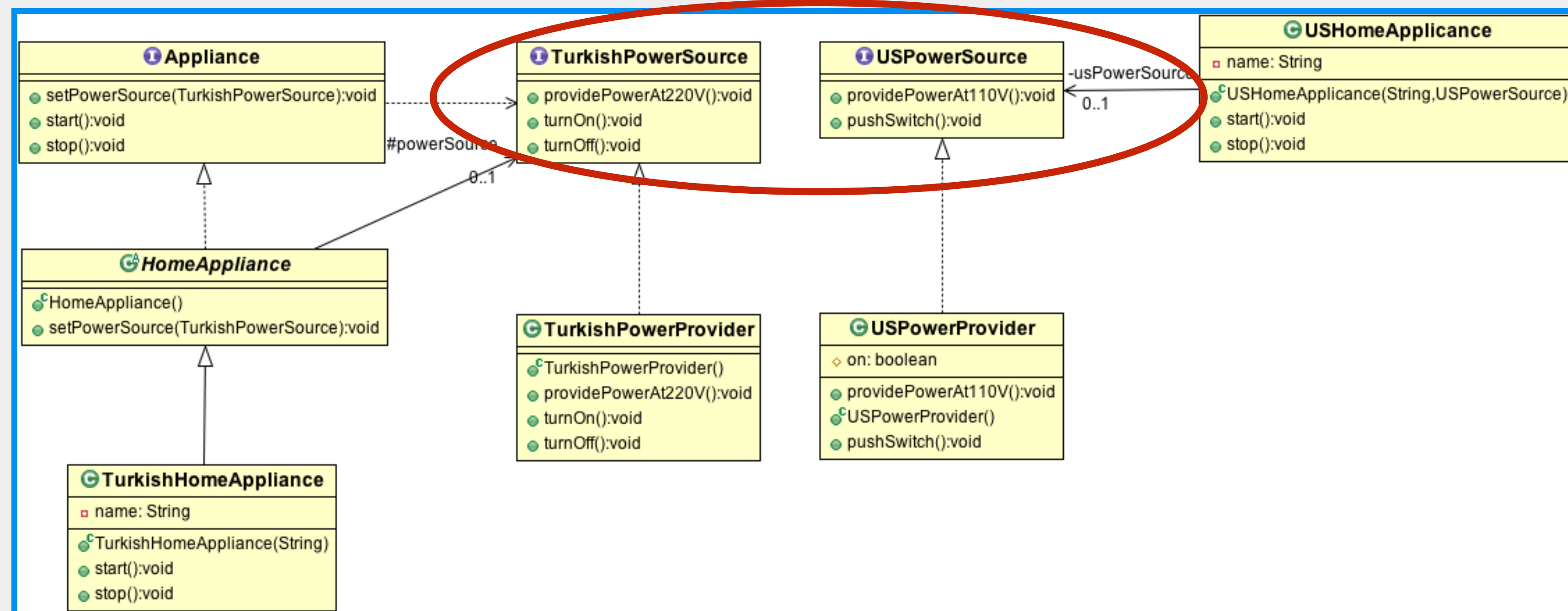


- `electricity.domain`
- Consider the same situation in US:
 - Electric household appliances and the infrastructure that powers them in the United States (US).





- An electrical appliance manufactured in Turkey can't operate in the US, and vice versa.
- This is because they are powered by incompatible power interfaces.





- `electricity.problem1`
- To overcome the problem of the incompatible interface, changes can be made to the **TurkishHomeAppliance** class.
- In this case, the **TurkishHomeAppliance** object would need to be modified for each new power source.

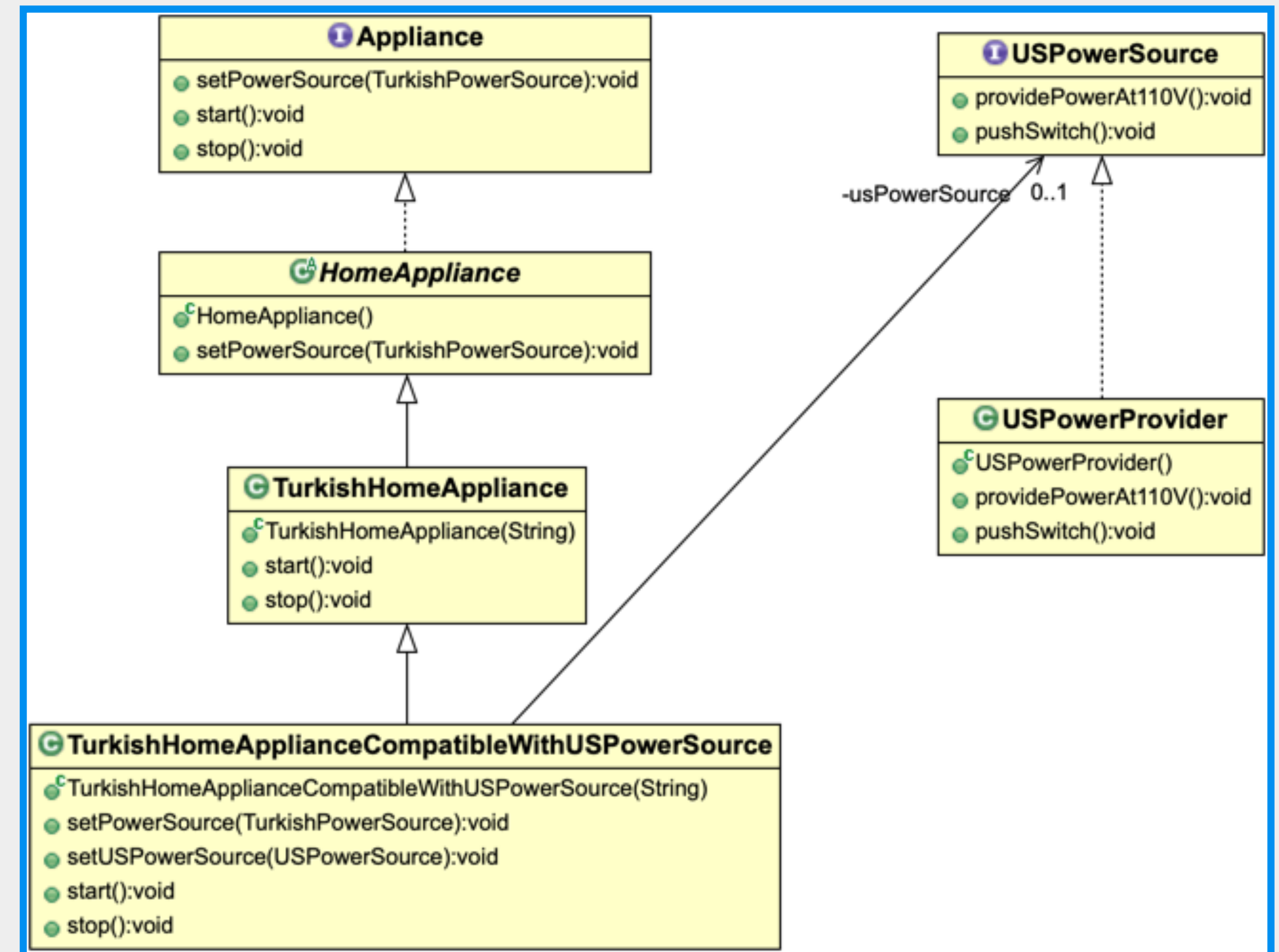
Problem - II



- Making changes to the class with the incompatible interface would violate the Open/Closed Principle (OCP).
- Moreover, if the code for the **TurkishHomeAppliance** class is unavailable, this change cannot be made anyway.
- As an alternative, the change can be made by extending the **TurkishHomeAppliance** class through inheritance and overriding its methods.



- `electricity.problem2`
- Changes can be made in a subclass of the **TurkishHomeAppliance** class to overcome the incompatible interface problem.



Problem - III



- However, a separate subclass is required for each incompatible situation in this case.
- This may not be a major problem in itself.
- The real problem lies elsewhere.

Problem - IV



- The fundamental difficulty in resolving the incompatibility issue through inheritance lies in embedding the adapter within the electrical device.
- This creates a situation that goes against SRP.
- However, adaptability is a separate responsibility and role, so it should not be embedded within the main entity.

Problem - V



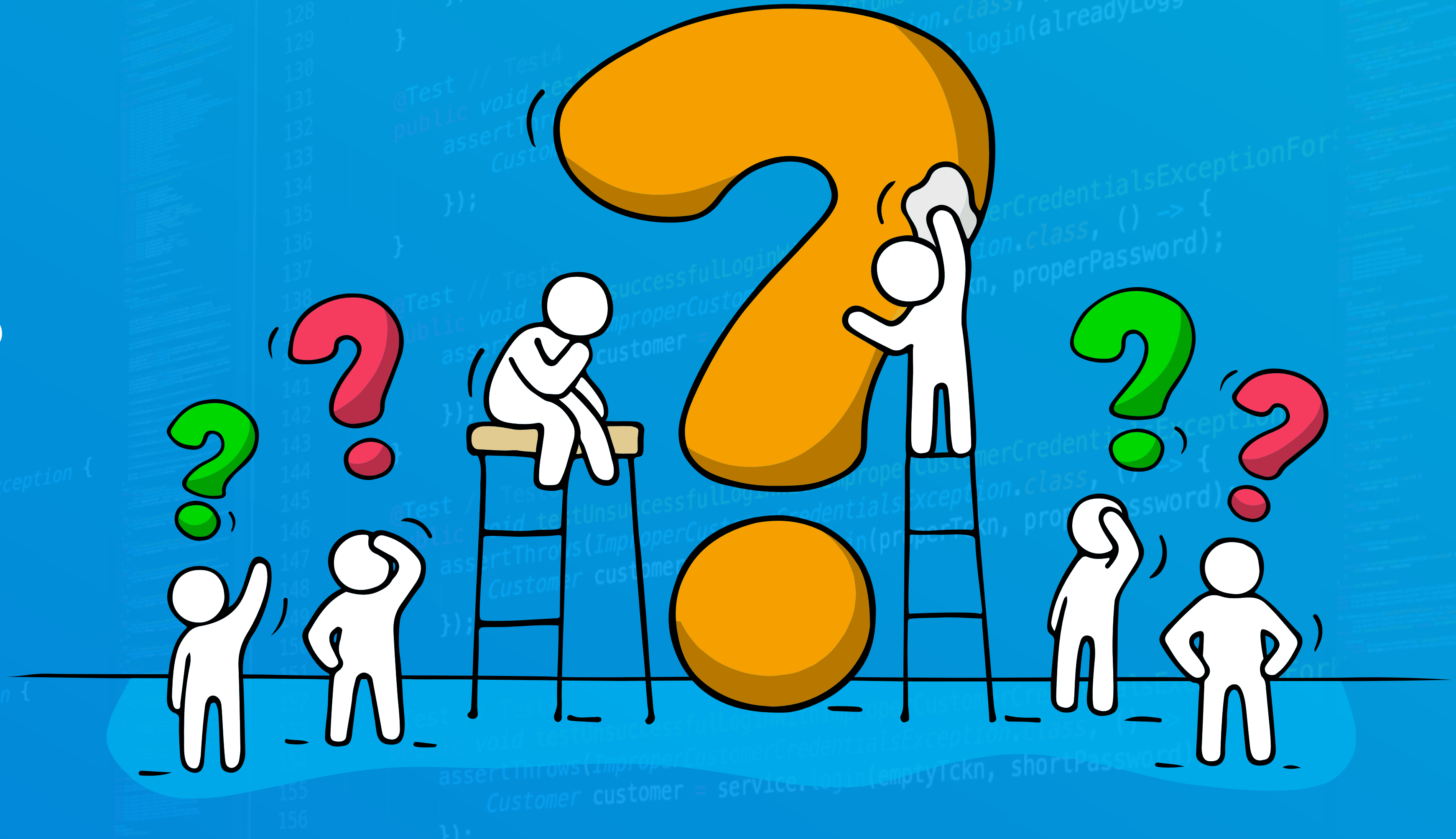
- Additionally, if this is done, separate adapters must be embedded inside each device.
- When the same device is shipped to multiple countries, a different built-in adapter is required for each different standard.

Problem - VI



- Another difficulty with solving the incompatibility problem by extension is when the existing electrical device cannot be extended.
- final/sealed classes
- Even if the device is expandable, it may need to be replaced in new versions due to the expanding class's high dependency.

Questions?



Solution



Adapter



- **Intent:** To convert the interface of a class into one that clients expect. The Adapter allows classes to work together that would otherwise be incompatible.
- It is also called wrapper.
- The adapter is known as 'adaptör' in Turkish.
- It is one of the most easily understandable structural patterns.
 - But its variations may make it complicated.

Intent



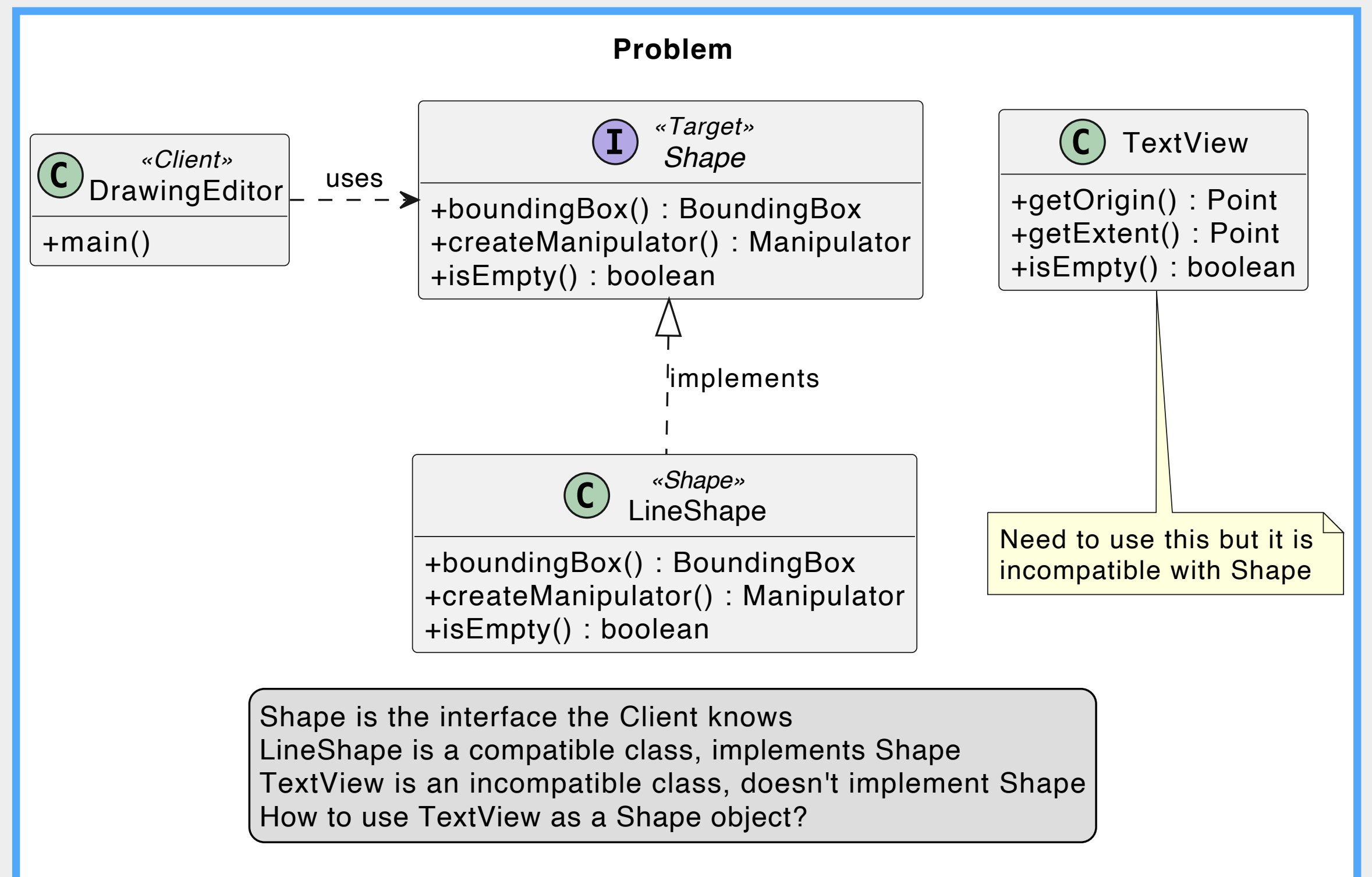
- The solution is to resolve the compatibility issue within a compatible class.
- This class enables existing clients to work with classes that have incompatible interfaces.
- This class is an **Adapter**.
- The adapter class acts as an intermediary between the incompatible class (adaptee) and the interface with which the client is familiar, thereby making the incompatible class compatible with that interface.



- **GoF**, p: 139, ...
 - Motivation
 - Applicability
 - Structure
 - Participants
 - Collaborations

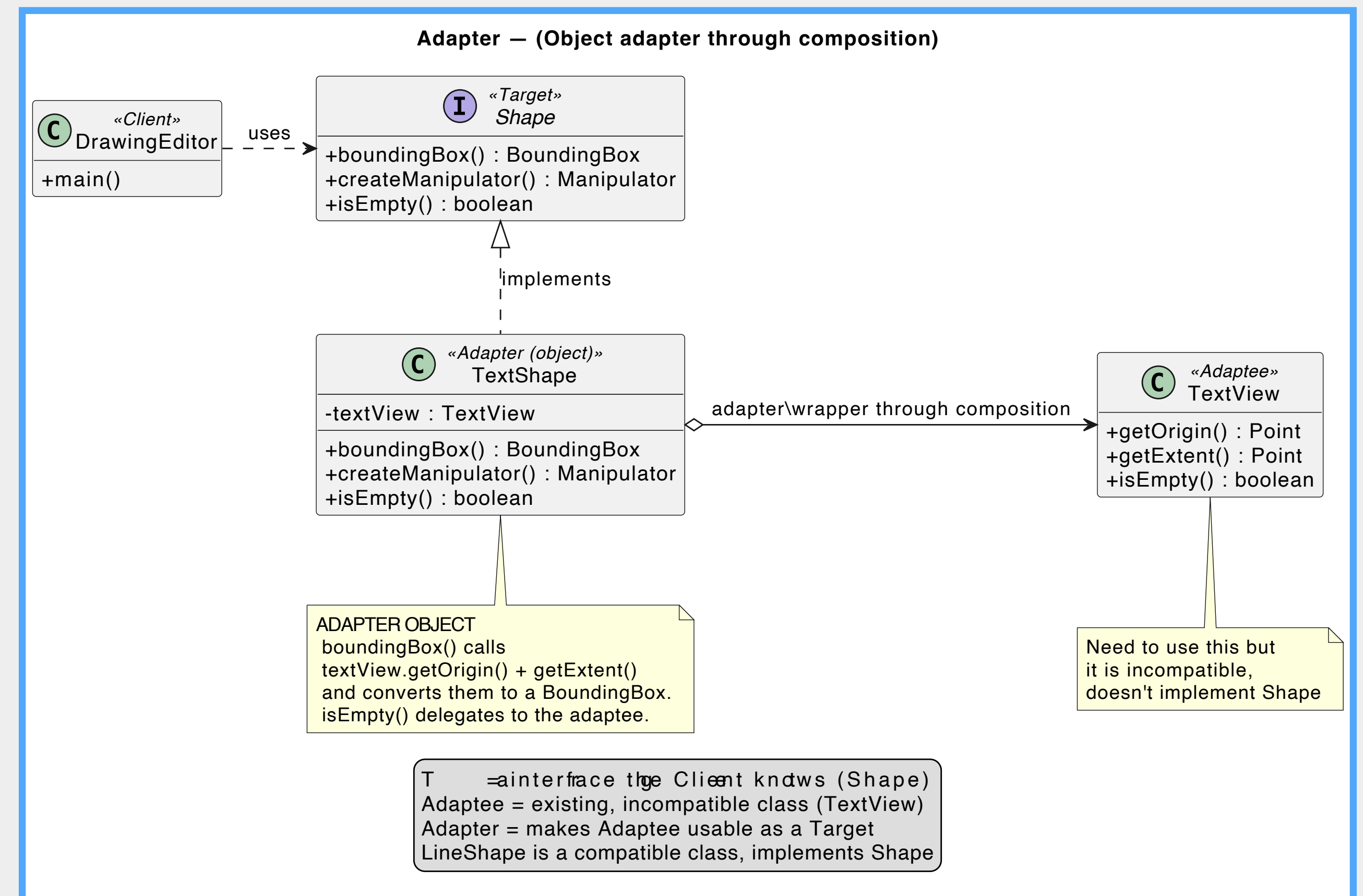


- gof
- Here **Shape** is interface the client, **DrawingEditor**, is familiar with.
- **LineShape** is a **Shape** so **DrawingEditor** can work with it.
- **DrawingEditor** needs to work with **TextView** that has an incompatible interface with **Shape**.





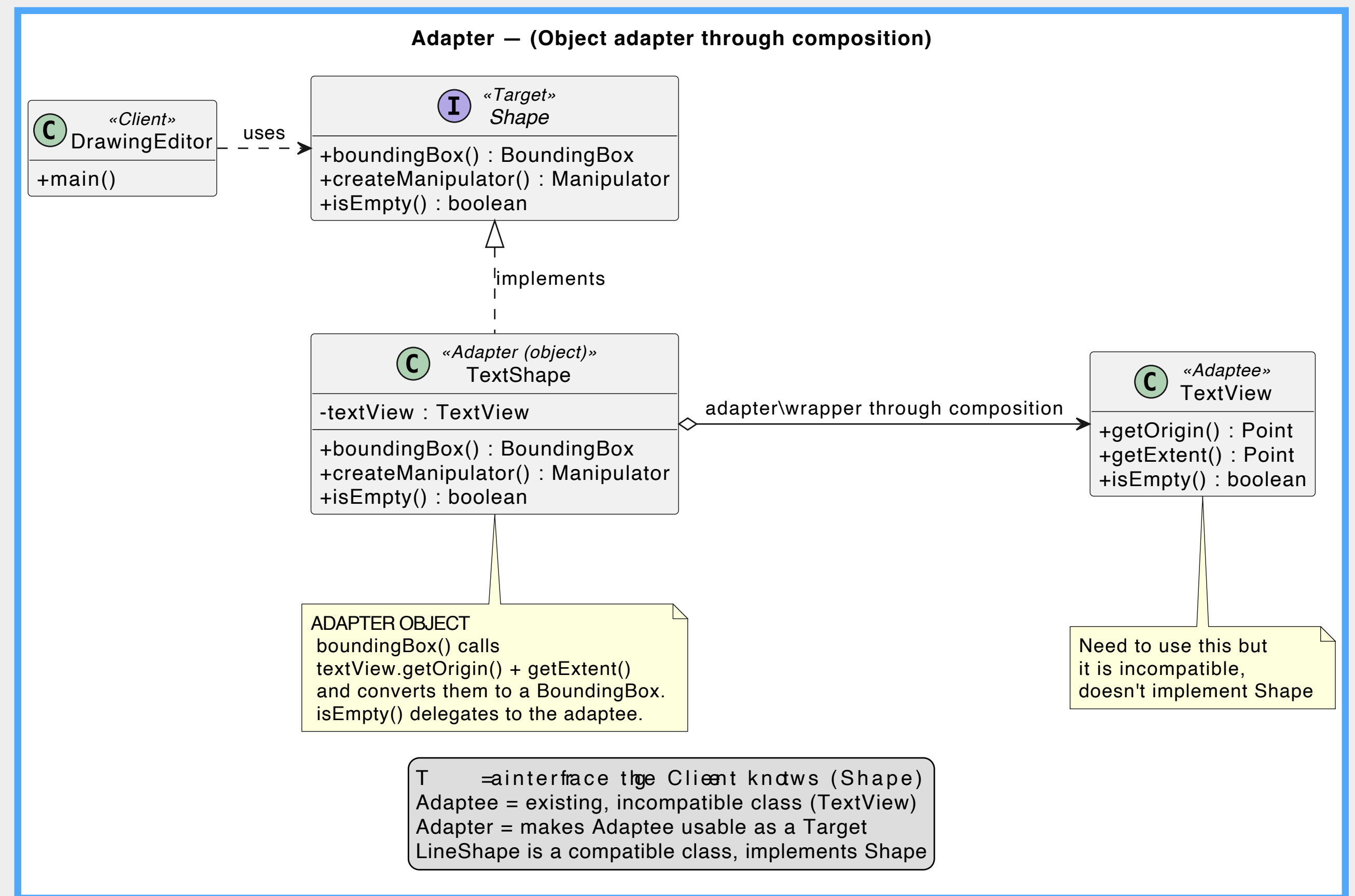
- gof
- **TextShape** is the adapter that adapts **TextView** to **Shape**.
- In this case **DrawingEditor** can work with **TextView** through **TextShape**.



Object Adapter - I



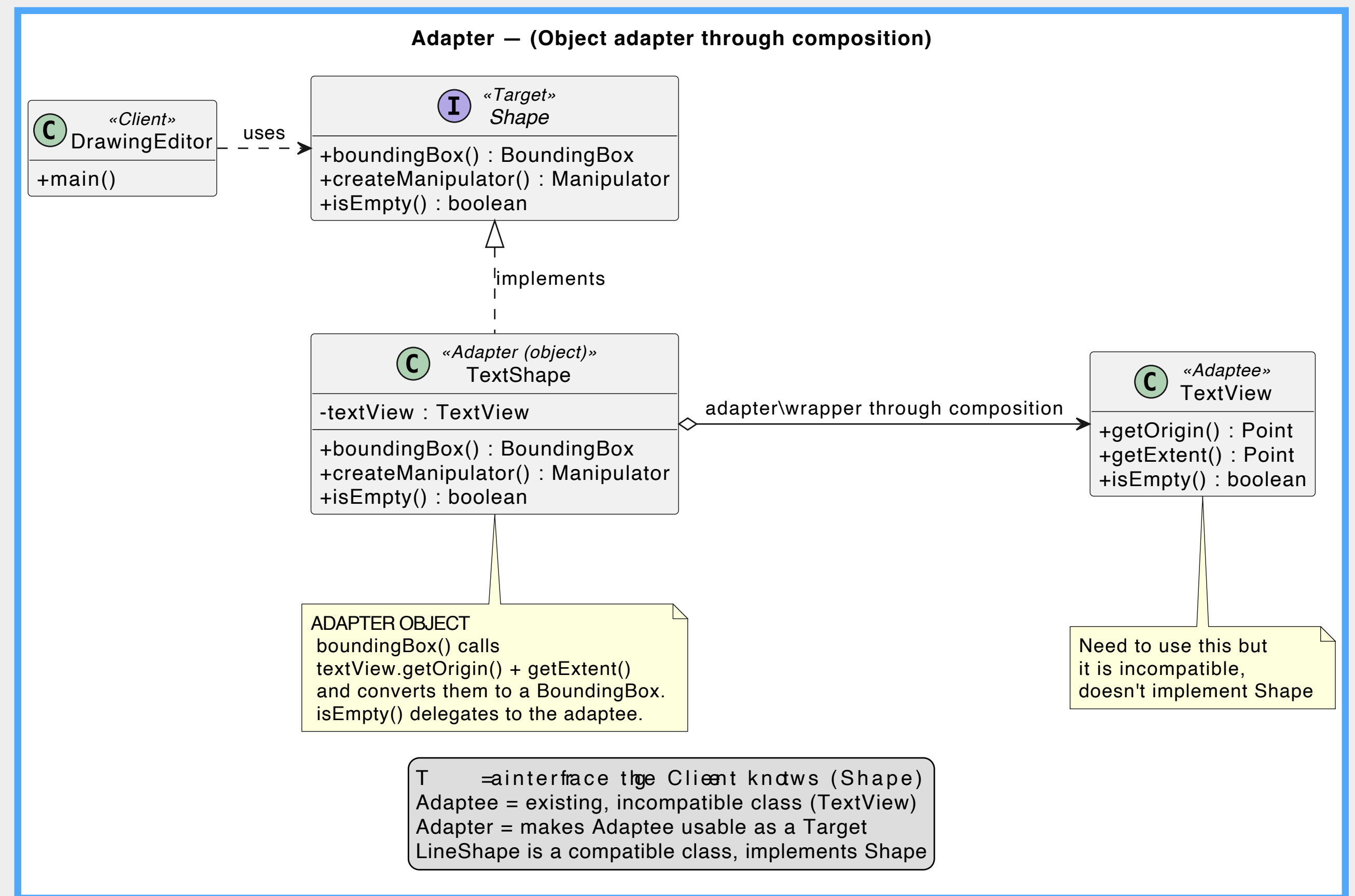
- GoF calls this implementation, **Object Adapter** because the adapter, **TextShape**, accesses **TextView**, the adaptee through the composition.
- So **TextShape** takes an object of **TextView** as its instance variable.



Object Adapter - II



- **TextShape** is a sub-type of **Shape** and implements it.
- **TextShape** conveys the calls that are made by the client to **TextView**, the incompatible object.



electricity.powerAdapter1

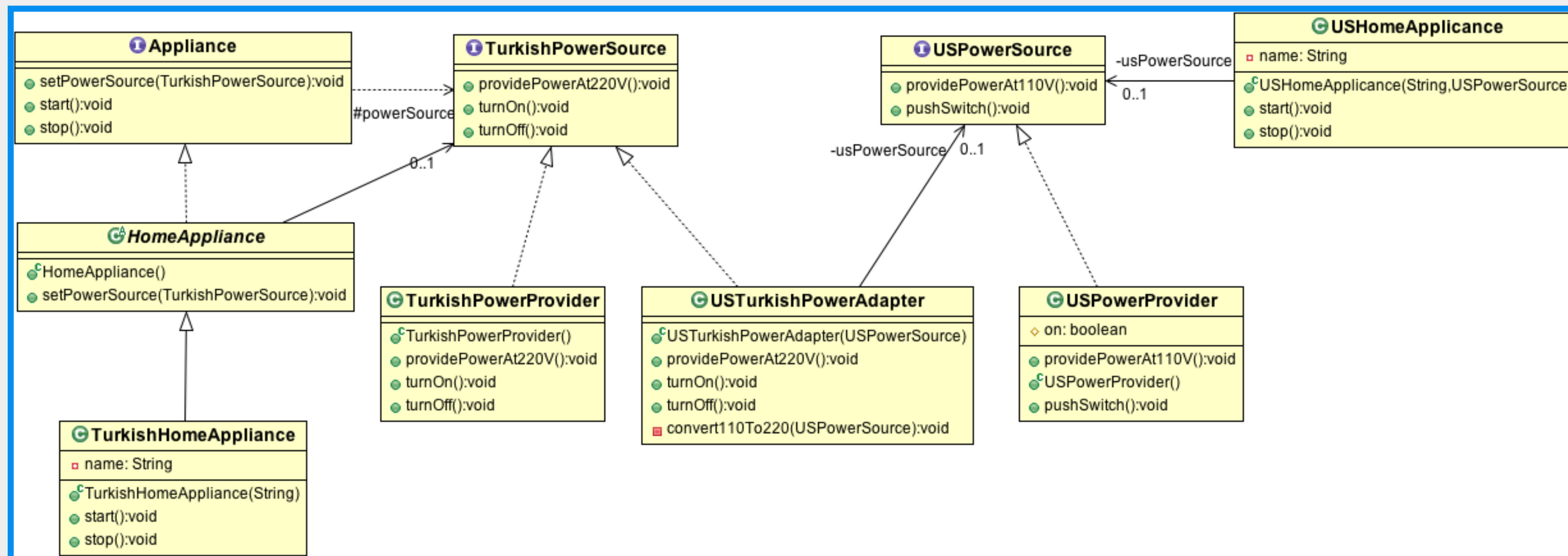


- `electricity.powerAdapter1`
- Think about working with home appliances suitable for Turkish power source in US.
- The solution is to create an adapter, `USTurkishPowerAdapter`, that implements the `TurkishPowerSource` interface and can communicate with the incompatible object, the `USPowerSource` through composition.
- In this case, `TurkishHomeAppliance` objects can work with `USPowerSource` via the `USTurkishPowerAdapter`.

electricity.powerAdapter1



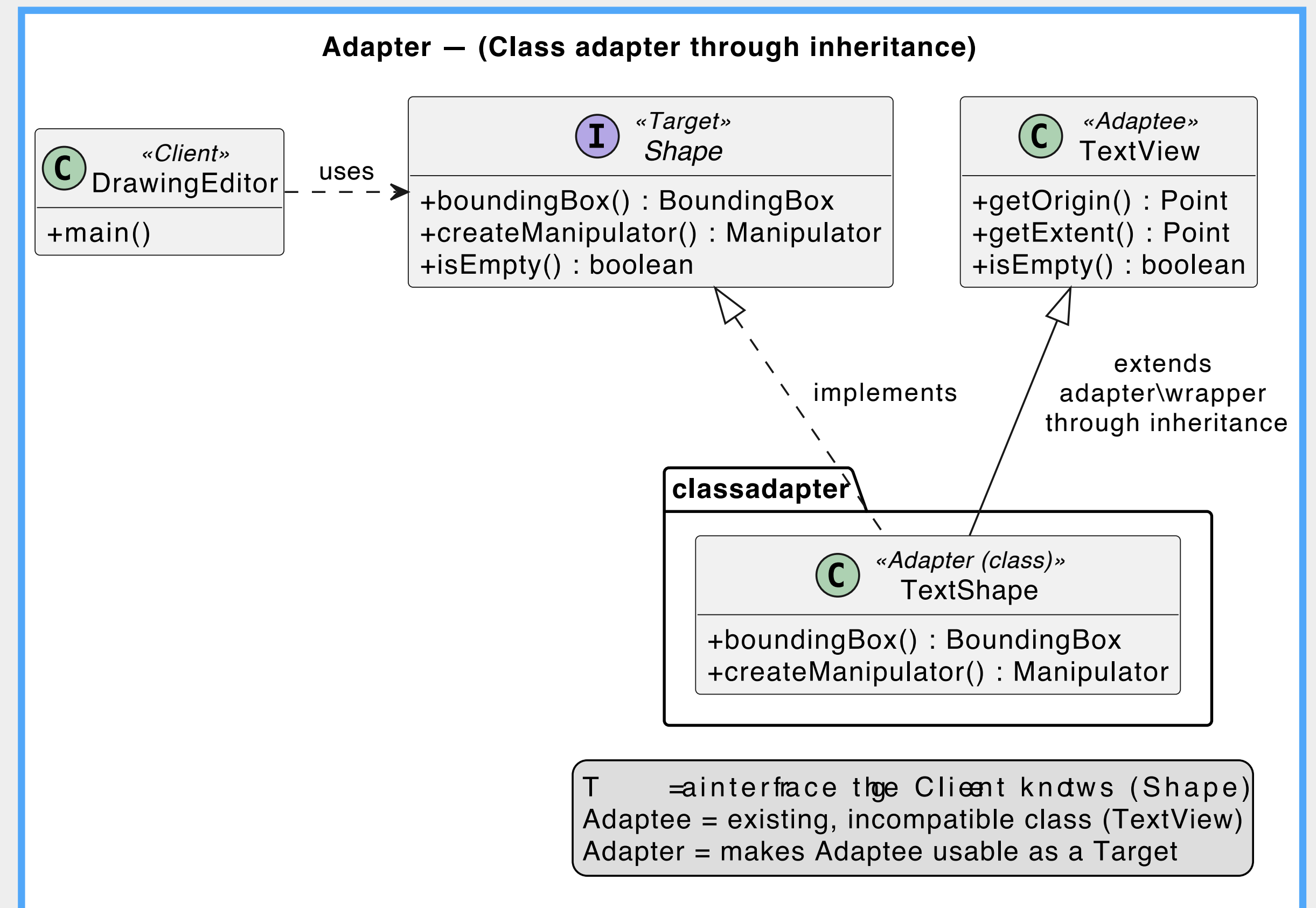
- The `USTurkishPowerAdapter` acts as a *wrapper* for the `USPowerSource`, adapting it to the `TurkishPowerSource` interface.
- This is an **Object Adapter** implementation.



Class Adapter - I



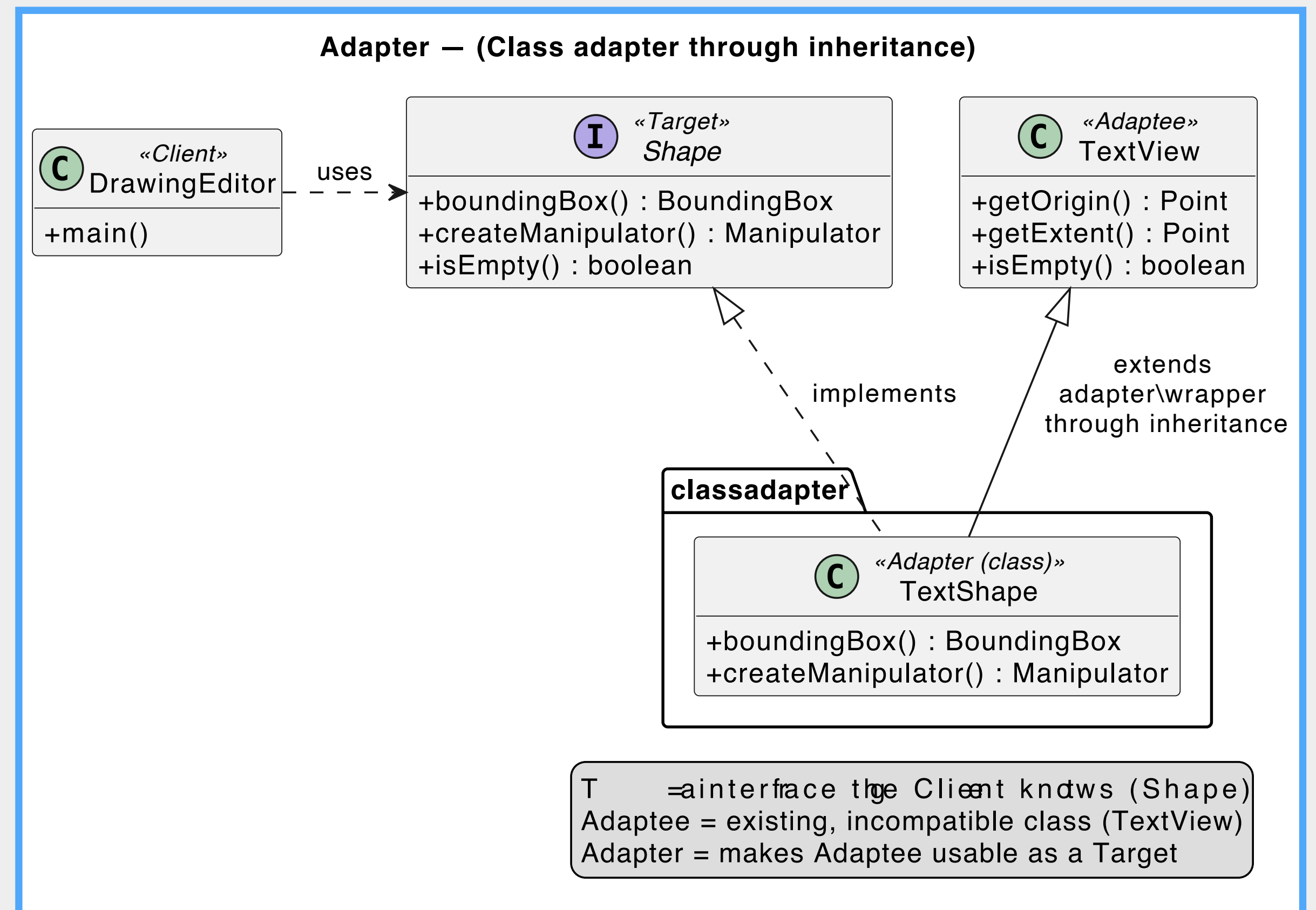
- GoF mentions another form of **Adapter** pattern that it calls **Class Adapter**.
- In Class Adapter, the adapter, **TextShape** extends the adaptee, **TextView**, the object with the incompatible interface.



Class Adapter - II

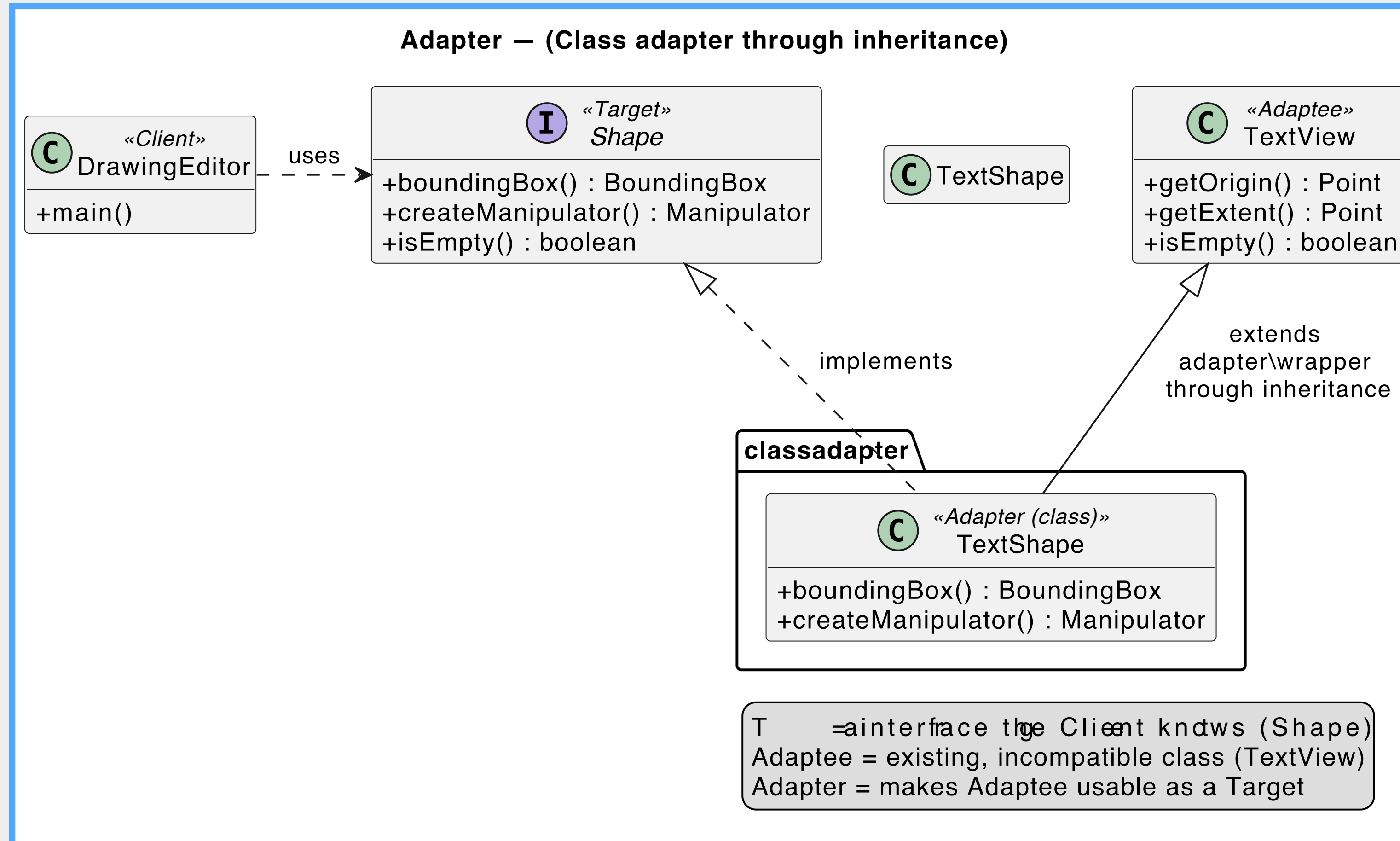


- In **Class Adapter** the adapter commits to a specific adaptee through inheritance; so for each different adaptee there needs to be another adapter class that is extending the adaptee.





• gof



electricity.classAdapter



- `electiricity.classAdapter`

Class Adapter vs. Object Adapter - I



- **Class Adapter** uses inheritance through sub-classing (is-a) whereas **Object Adapter** uses composition (has-a).
- **Object Adapter** is more flexible because it can work with the sub-types of adaptee where **Class Adapter** provides adaptation service only for a specific adaptee class.
- In **Class Adapter** the relationship between the adapter and adaptee is established at compile time through their classes.
 - That's why GoF calls it **Class Adapter**.

Class Adapter vs. Object Adapter - II

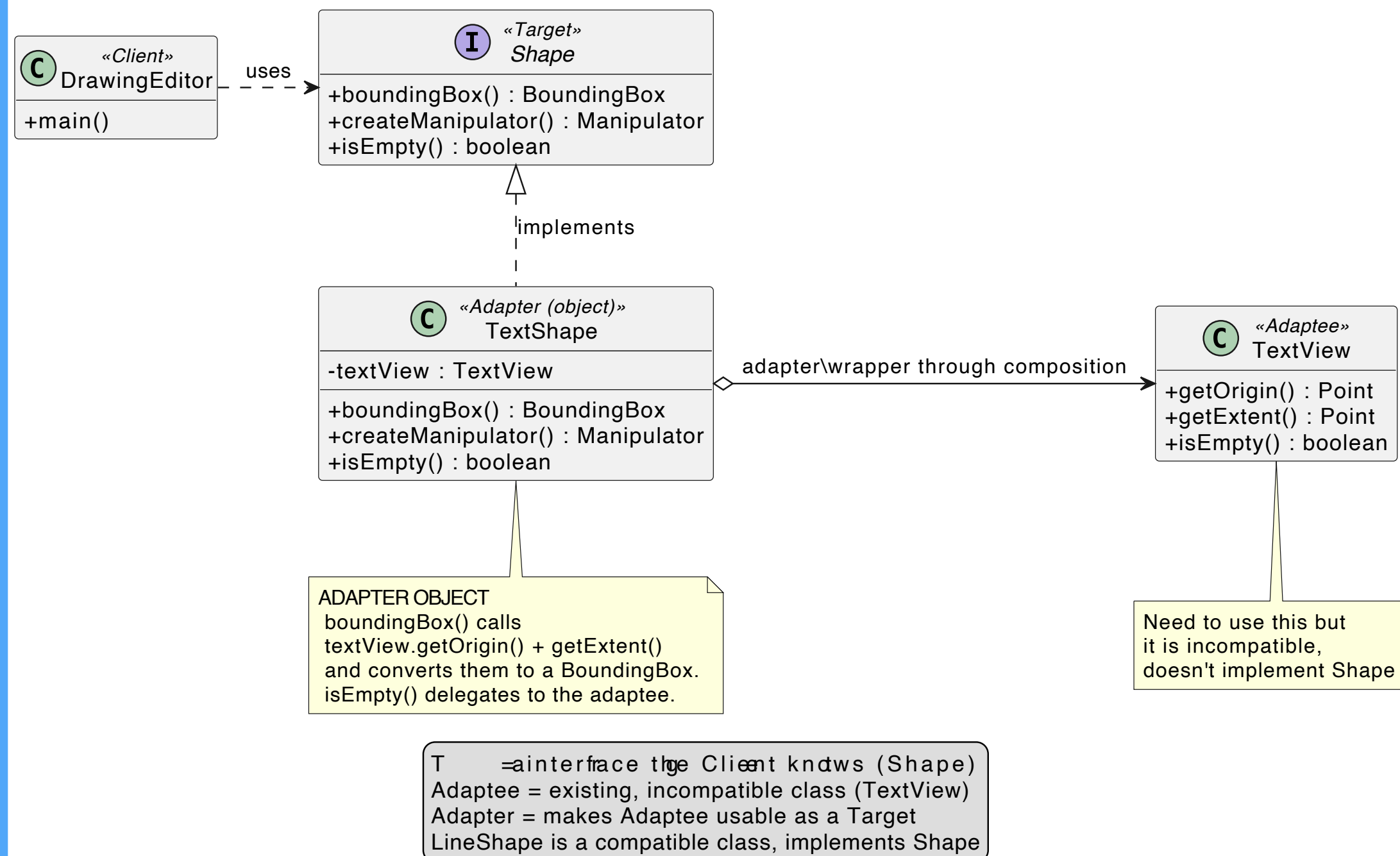


- Prefer the **Object Adapter** unless you specifically need to override adaptee behaviours:
- The **Object Adapter** is more flexible and survives final/sealed classes.
- **Object Adapter** vs. **Class Adapter** tradeoff is a matter of *favoring object composition over class inheritance*, which is the second principle of Gof, mentioned in page 20.

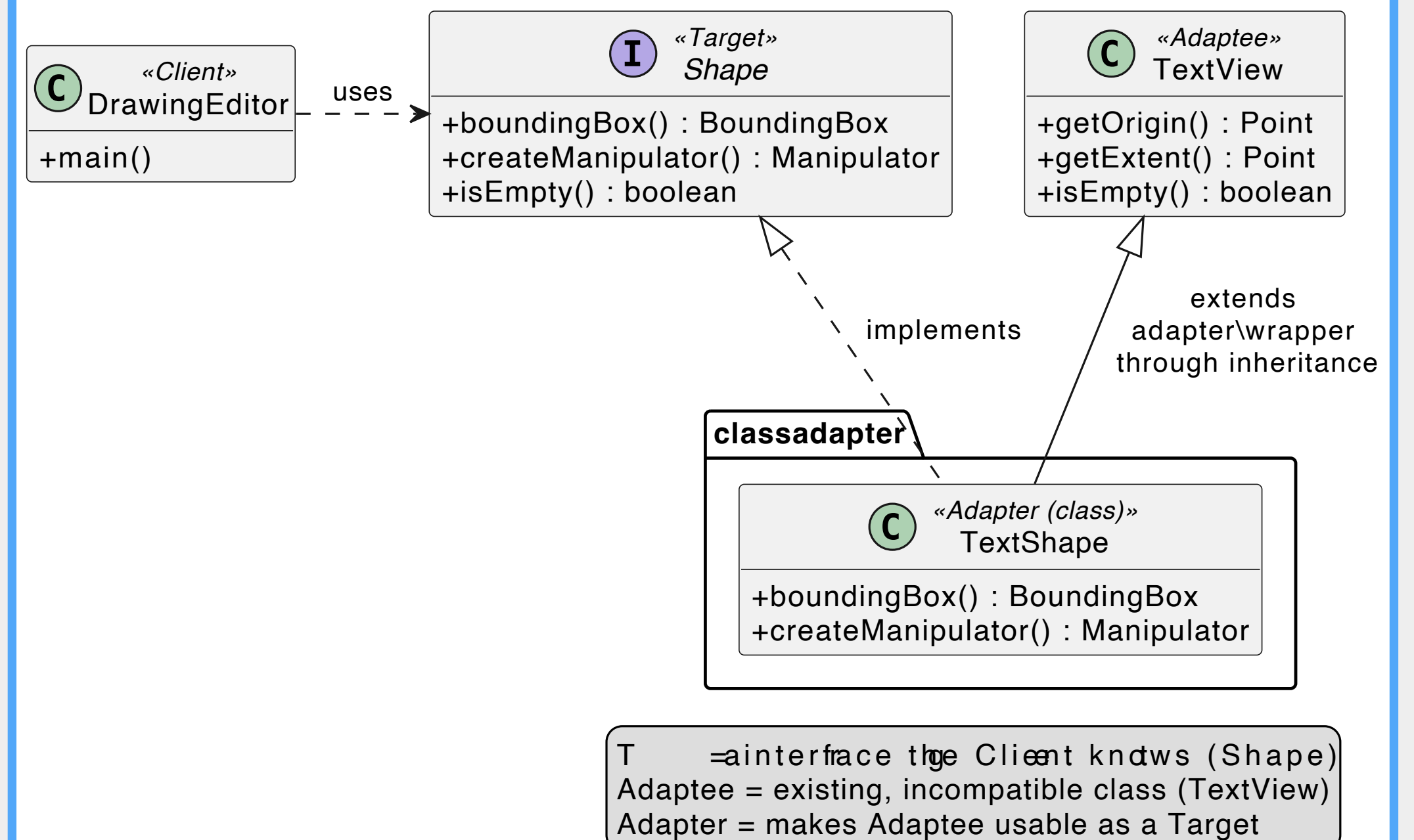
Class Adapter vs. Object Adapter - II



Adapter — (Object adapter through composition)



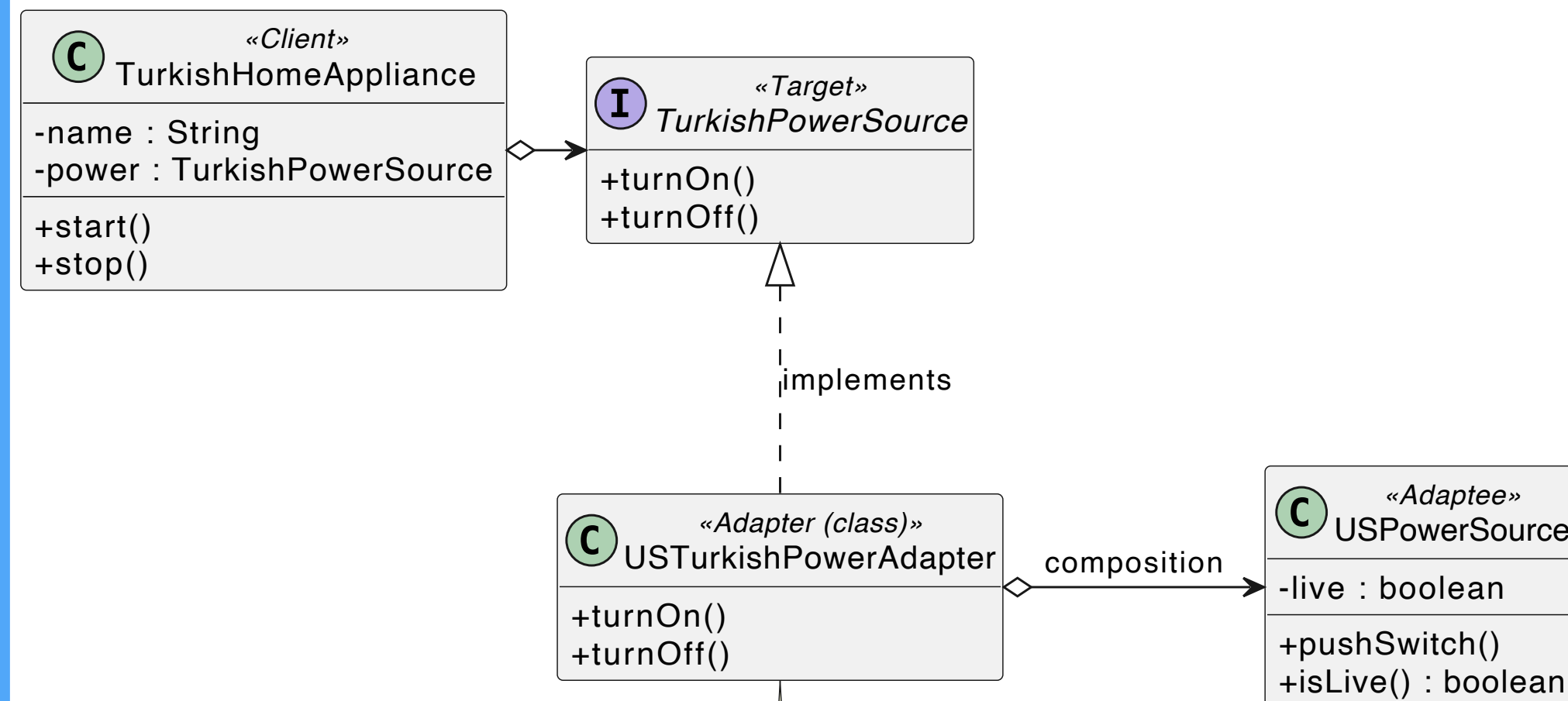
Adapter — (Class adapter through inheritance)



Class Adapter vs. Object Adapter - IV



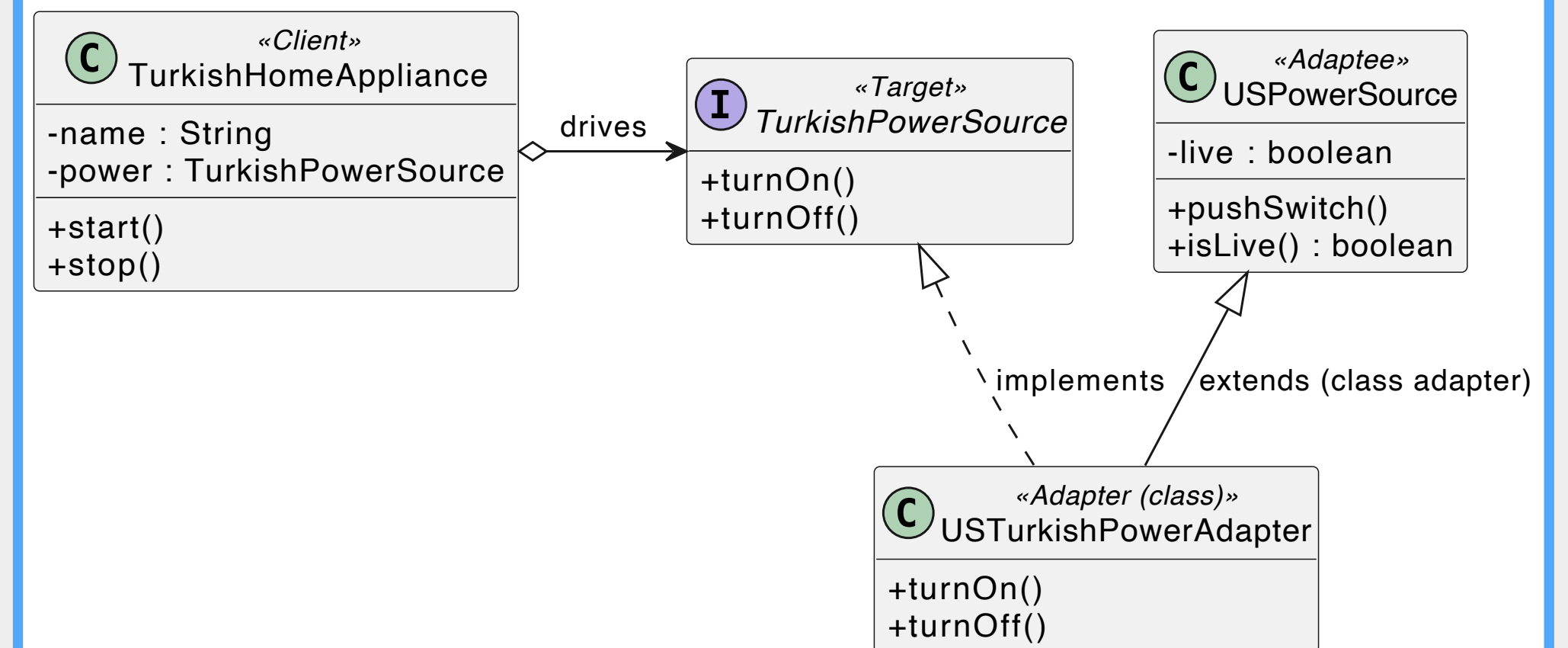
Electricity Object Adapter



Object Adapter
Reaches the adaptee, takes it as an instance variable.
It delegates `turnOn()/turnOff()` calls to `pushSwitch() / isLive()`.
Using `USPowerSource`, the adapter can also be used anywhere any of `USPowerSource`'s sub-types is expected.

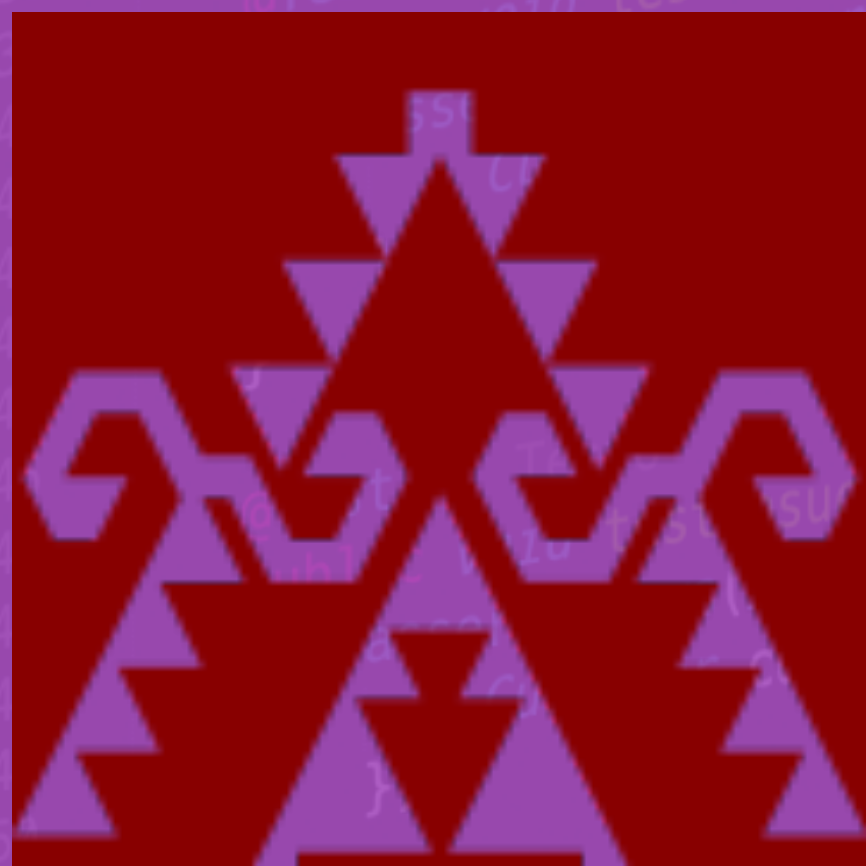
T = aTurkishPowerSource (turnOn / turnOff)
Adaptee = USPowerSource (pushSwitch / isLive)
Adapter = USTurkishPowerAdapter

Electricity Class Adapter



Class Adapter
Extends the Adaptee, implements the Target.
No adaptee field, no delegation:
`turnOn()/turnOff()` call the inherited `pushSwitch() / isLive()`.
Being a subclass of `USPowerSource`, the adapter can also be used anywhere a `USPowerSource` is expected, and could override its behaviour.

T = aTurkishPowerSource (turnOn / turnOff)
Adaptee = USPowerSource (pushSwitch / isLive)
Adapter = USTurkishPowerAdapter (is-a Adaptee and is-a Target)



Variations

Variations - I



- How much work adapters do depends on the problem.
- Sometimes adapters just delegate calls to the methods on the adaptee, which is the simplest case.
- Sometimes, adapters do more than just making incompatible objects compatible.
- In such cases, the adapter can provide certain services either before or after accessing the adaptee, incompatible object.

Variations - II

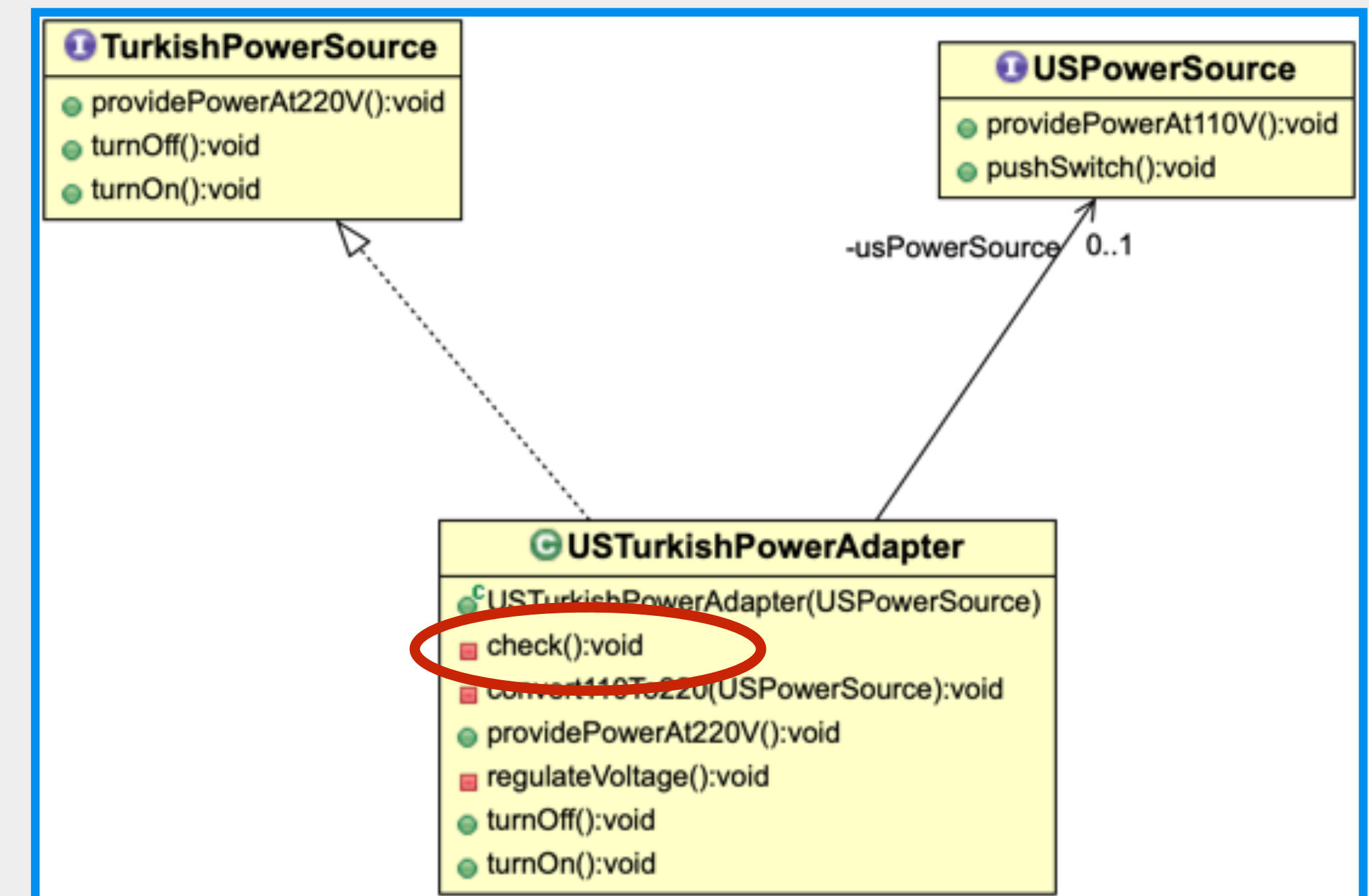


- It is important that the adapter doesn't lose focus by performing services that are not closely related to compatibility.
- If additional services would distract the adapter, it is better to move them to separate structures.

electricity.powerAdapter2



- `electricity.problem2`
- This solution uses the **USTurkishPowerAdapter** to display and adjust power-related information, such as voltage and frequency, in the **check()** method.
- This is done for security purposes before transmitting power to the **TurkishHomeAppliance** object.



Two-Way Adapter - I



- So, how can we ensure that the `USHomeAppliance` object works in Turkey?
- How should we modify the `USTurkishPowerAdapter` object so that the `USHomeAppliance` object can work with the `TurkishPowerSource`?

Two-Way Adapter - II

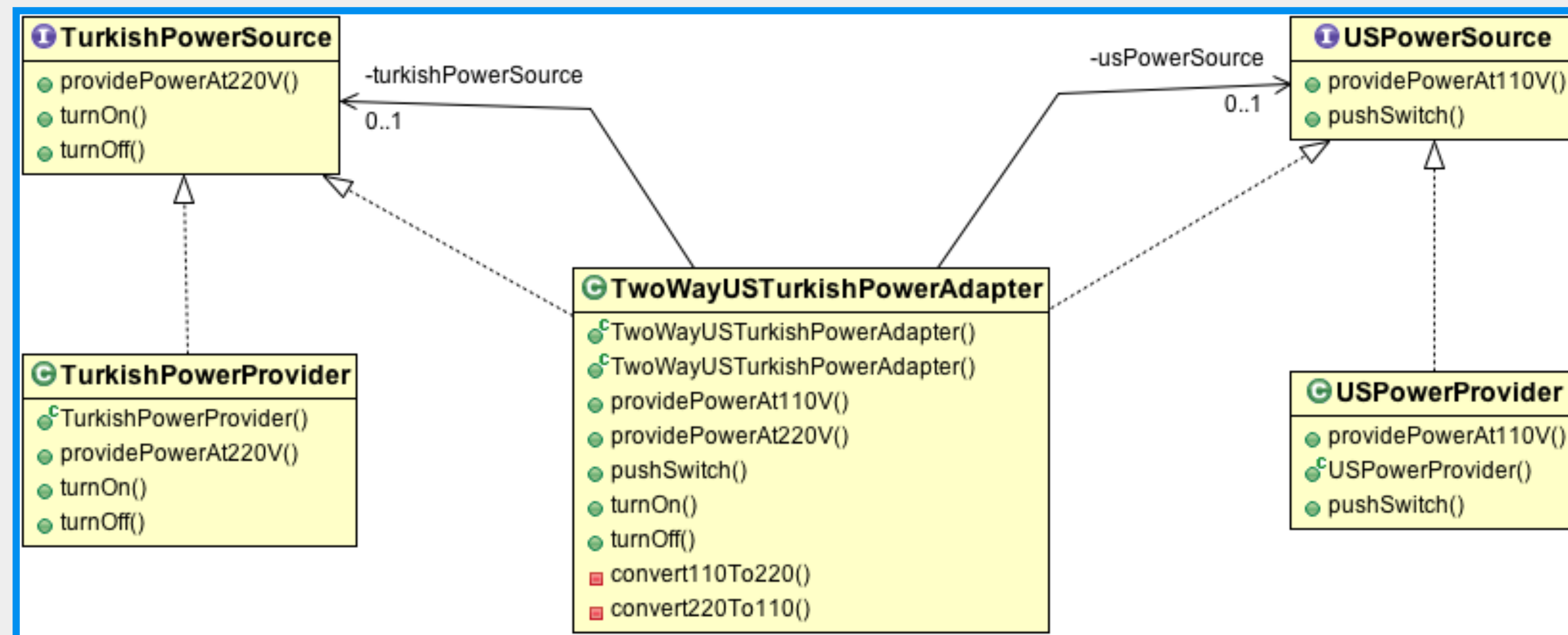


- In this case, rather than modifying the `USTurkishPowerAdapter` object, we could create a two-way adapter object:
`TwoWayUSTurkishPowerAdapter`
- `TwoWayUSTurkishPowerAdapter` object would function as a two-way adapter.

electricity.twoWayAdapter



- `electricity.twoWayAdapter`
- The `TwoWayUSTurkishPowerAdapter` has two interfaces, making it a two-way adapter.



A Universal Adapter



- So far we need to create another adapter for each incompatible interface i.e. adaptee.
- Can we have just one adapter, a universal adapter, which is suitable for many different adaptees?
- If we implement the universal adapter as an **Object Adapter** then it should accommodate all different incompatible interfaces; which would make it very complex.
- Otherwise we need a new adapter for each new incompatible interface.

Pluggable Adapters - I



- Or maybe we can think of another approach to get a universal adapter.
- How can we have just one adapter for many different adaptees without making the adapter too complicated?
- Can we think of an adapter that is pluggable to different kinds of adaptees?
- So the adaptor doesn't commit to any specific adaptee but would be pluggable to accommodate any incompatible interface.

Pluggable Adapters - II



- For this purpose in page 142, GoF says: *A class is more reusable when you minimize the assumptions other classes must make to use it.*
- An adapter is more reusable when it makes fewer assumptions about the interfaces of different adaptees.
- How can we make the adapter to have minimum assumptions about the interfaces of different adaptees?

Pluggable Adapters - III



- Can we make sure that all adaptees have the same interface?
 - Probably not!
- So building the adaptation into a small, reusable adapter driven by a narrow interface would make it re-usable.
- If the adaptees have a narrow interface it would be much easier to make it a good fit the pluggable adapter because the pluggable adapter would be easier to implement.

Pluggable Adapters - IV



- GoF in page 144 lists three different ways to implement pluggable adapters:
 - Using abstract operations
 - Using delegate objects
 - Parameterized adapters

Abstract Operations - I



- **Using abstract operations:** *It is a plug in by inheritance.*
- Create an abstract class with an abstract operation for the adapter and each adapter would extend the class and overrides the operation according to different power systems.
- **PowerAdapter** also utilizes **Template Method** pattern.

```
abstract class PowerAdapter implements TurkishPowerSource {  
    public final void turnOn() { deliver(); }  
    public final void turnOff() { cut(); }  
    protected abstract void deliver();  
    protected abstract void cut();  
}
```

Abstract Operations - II



- Abstract methods of the **PowerAdapter** would be overridden by each new adapter.

```
abstract class PowerAdapter implements TurkishPowerSource {  
    public final void turnOn() { deliver(); }  
    public final void turnOff() { cut(); }  
    protected abstract void deliver();  
    protected abstract void cut();  
}
```

```
class USTurkishPowerAdapter extends PowerAdapter {  
    private final USPowerSource usPowerSource;  
  
    public USTurkishPowerAdapter(USPowerSource usPowerSource){  
        this.usPowerSource = usPowerSource;  
    }  
  
    protected void deliver() {  
        usPowerSource.pushSwitch();  
    }  
  
    protected void cut() {  
        usPowerSource.pushSwitch();  
    }  
}
```

```
class UKTurkishPowerAdapter extends PowerAdapter {  
    private final UKPowerSource ukPowerSource;  
  
    public UKTurkishPowerAdapter(UKPowerSource ukPowerSource){  
        this.ukPowerSource = ukPowerSource;  
    }  
  
    protected void deliver() {  
        ukPowerSource.flipToggle();  
    }  
  
    protected void cut() {  
        ukPowerSource.flipToggle();  
    }  
}
```


Abstract Operations - III

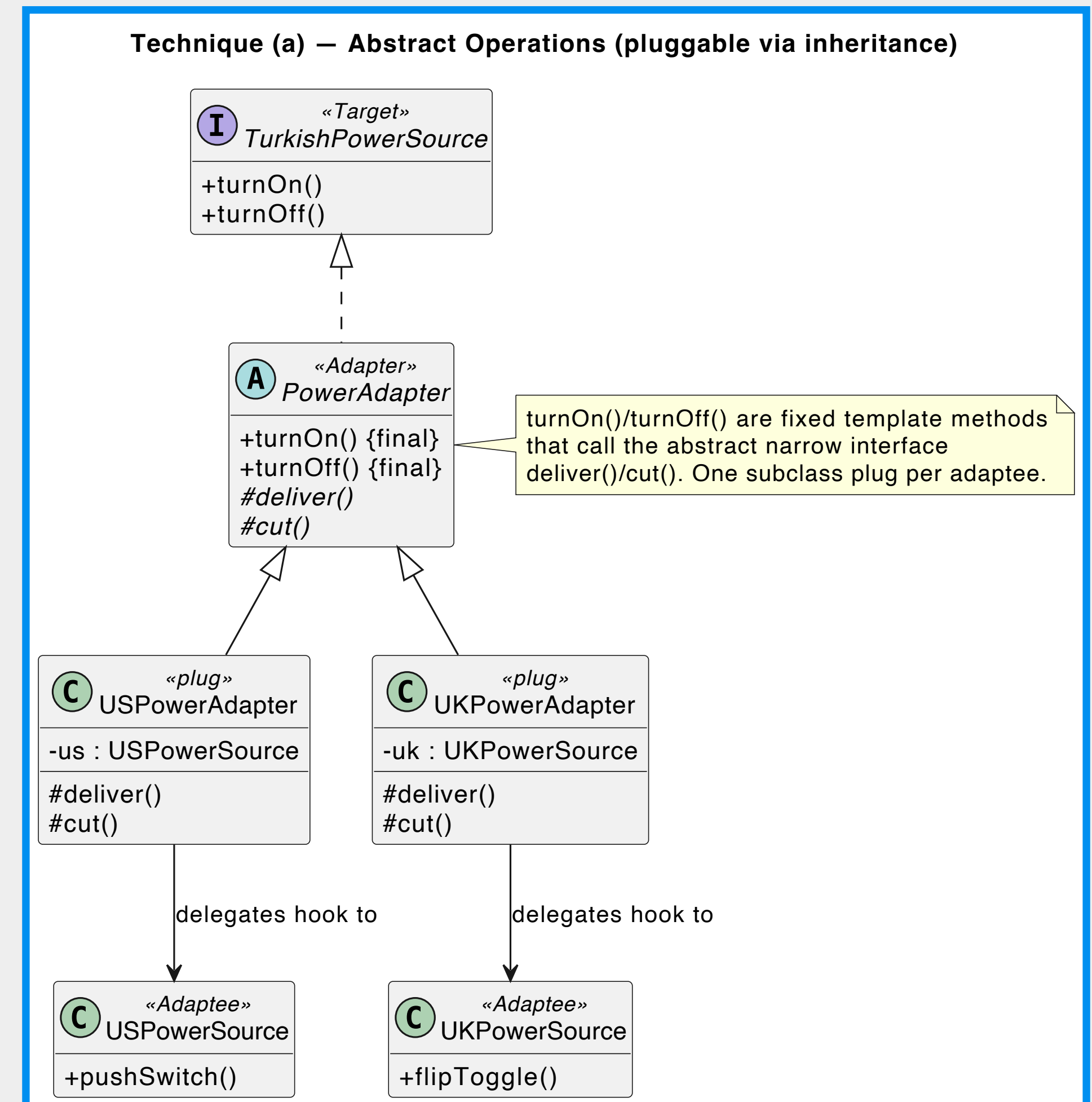


- The methods in `PowerAdapter` that would delegate the operations to adaptees are declared abstract and overridden in its subtypes such as `USTurkishPowerAdapter` or `UKTurkishPowerAdapter`.
- So the solution uses the **Object Adapter** but makes it the adapter to accommodate different adaptees for different power systems.
- So it is a plug in by composition.
- Inheritance is used for different adapters classes.

electricity.pluggable.abstractOperations



- electricity.pluggable.
abstractOperations



Delegate Object - I



- **Using delegate object:** *It is plug in by composition.*
- The adapter forwards operations to a swappable delegate of type **PowerDelivery**, which is the unifying interface for different power systems.

```
final class DelegatingPowerAdapter implements TurkishPowerSource {  
    private final PowerDelivery delivery;  
  
    public DelegatingPowerAdapter(PowerDelivery delivery) {  
        this.delivery = delivery;  
    }  
  
    public void turnOn() { delivery.deliver(); }  
  
    public void turnOff() { delivery.cut(); }  
}
```

```
public interface PowerDelivery {  
    void deliver();  
    void cut();  
}
```


Delegate Object - II



- This is the preferred solution that applies the principle *favor object composition over class inheritance*.
- **PowerDelivery** objects reaches specific power sources through composition.

```
public interface PowerDelivery {  
    void deliver();  
    void cut();  
}
```

```
public final class USPowerDelivery implements PowerDelivery {  
    private final USPowerSource us;  
  
    public USPowerDelivery(USPowerSource us) { this.us = us; }  
  
    public void deliver() { us.pushSwitch(); }  
  
    public void cut() { us.pushSwitch(); }  
}
```

```
public final class UKPowerDelivery implements PowerDelivery {  
    private final UKPowerSource uk;  
  
    public UKPowerDelivery(UKPowerSource uk) { this.uk = uk; }  
  
    public void deliver() { uk.flipToggle(); }  
  
    public void cut() { uk.flipToggle(); }  
}
```

Delegate Object - III



- It is more flexible than the previous solution that uses inheritance.
- It has only one adapter, **DelegatingPowerAdapter**, which is a **final** class and pluggable for different **PowerDelivery** objects.
- There needs to be a new **PowerDelivery** class for each new different power system.

```
public interface PowerDelivery {  
    void deliver();  
    void cut();  
}
```

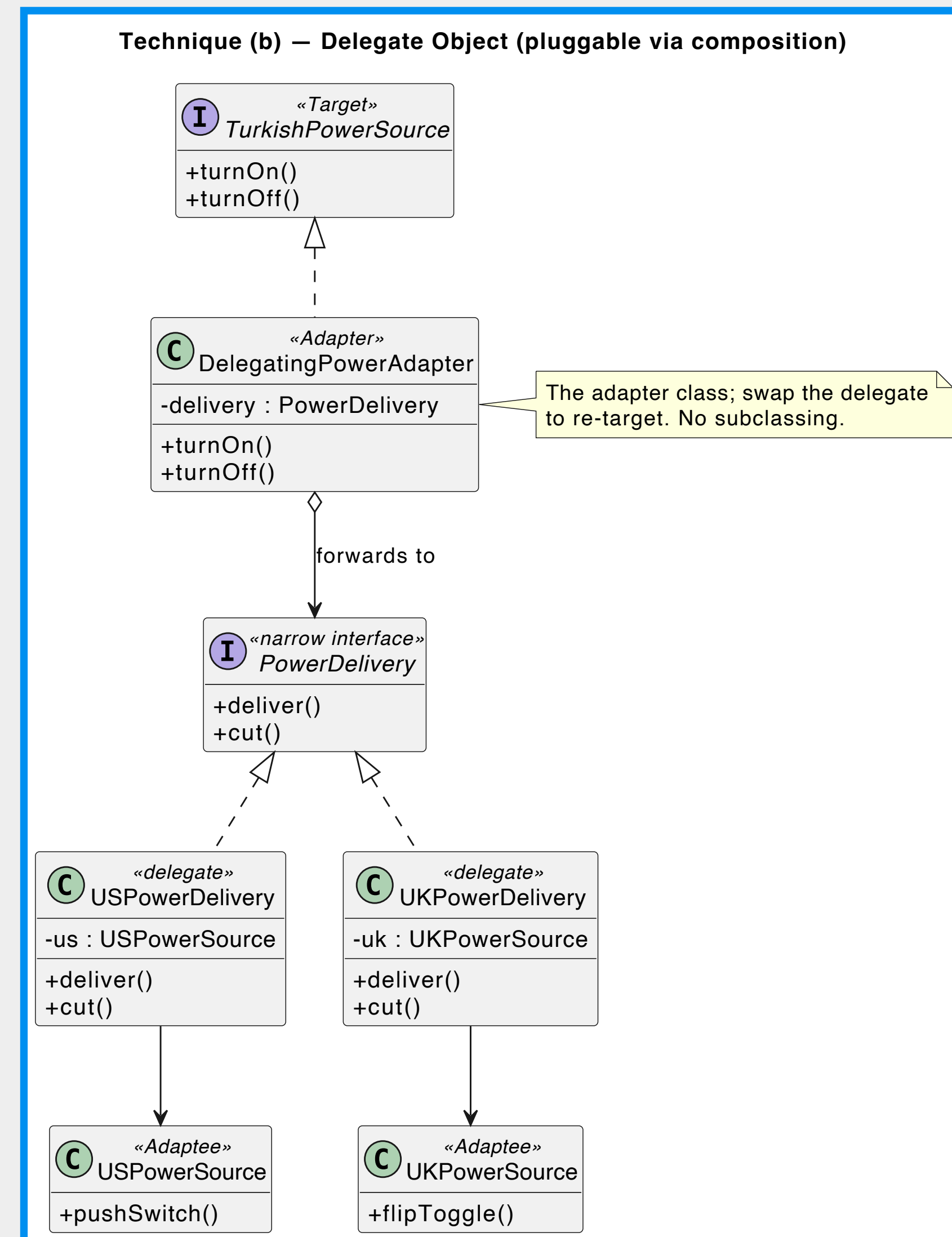
```
public final class USPowerDelivery implements PowerDelivery {  
    private final USPowerSource us;  
  
    public USPowerDelivery(USPowerSource us) { this.us = us; }  
  
    public void deliver() { us.pushSwitch(); }  
  
    public void cut() { us.pushSwitch(); }  
}
```

```
public final class UKPowerDelivery implements PowerDelivery {  
    private final UKPowerSource uk;  
  
    public UKPowerDelivery(UKPowerSource uk) { this.uk = uk; }  
  
    public void deliver() { uk.flipToggle(); }  
  
    public void cut() { uk.flipToggle(); }  
}
```


electricity.pluggable.delegateObject



- electricity.pluggable.delegateObject



Parameterized Adapters - I



- **Parameterized adapters:** *It is plug in by functions i.e. lambda expressions.*
- This is truly pluggable adapter that unifies the interface of different power systems under a tiny interface, **Runnable**.

```
final class PluggablePowerAdapter implements TurkishPowerSource {  
    private final Runnable onTurnOn;  
    private final Runnable onTurnOff;  
  
    public PluggablePowerAdapter(Runnable onTurnOn, Runnable onTurnOff) {  
        this.onTurnOn = onTurnOn;  
        this.onTurnOff = onTurnOff;  
    }  
  
    public void turnOn() { onTurnOn.run(); }  
  
    public void turnOff() { onTurnOff.run(); }  
}
```

```
public interface Runnable {  
    void run();  
}
```

Parameterized Adapters - II



- This is the more preferred solution that both applies the principle *favor object composition over class inheritance* and assumes only **Runnable** interface.
- **PluggablePowerAdapter** is a **final** class.
- **PowerDelivery** of the previous solution is replaced by **Runnable**.

```
public interface PowerDelivery {  
    void deliver();  
    void cut();  
}
```

```
public interface Runnable {  
    void run();  
}
```


Parameterized Adapters - III



- All power source systems can be reached through lambda expressions that is of type `Runnable`, as long as their operations would be in the form of `void f()` i.e. operation that receives nothing and returns `void`.
- The operations of different power systems are represented by `Runnable` now.

```
public interface Runnable {  
    void run();  
}
```

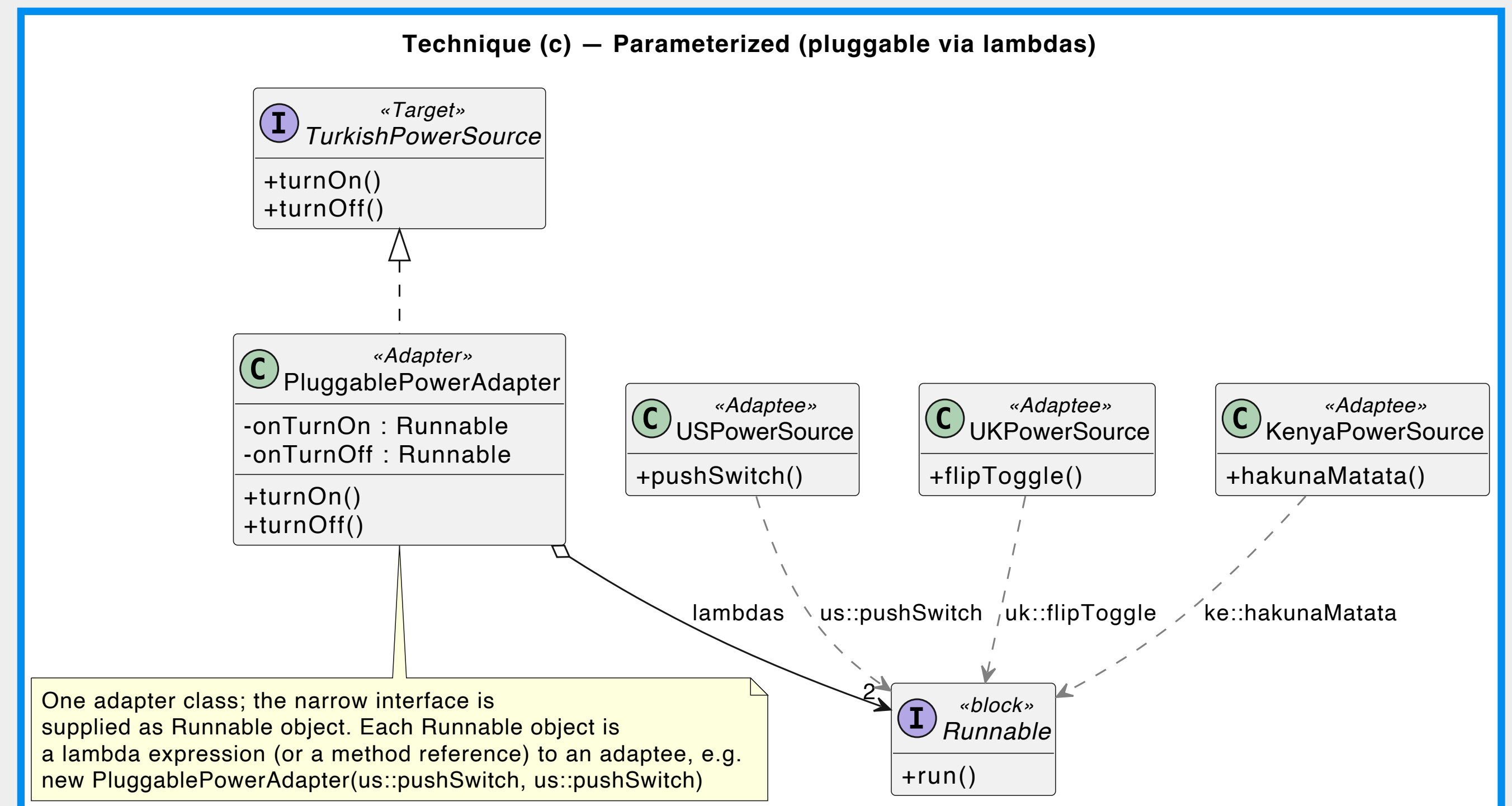
```
public final class USPowerSource {  
    public void pushSwitch() { .. }  
}
```

```
public final class UKPowerSource {  
    public void flipToggle() { .. }  
}
```

electricity.pluggable.parameterized



- electricity.pluggable.parameterized



Parameterized Adapters - IV



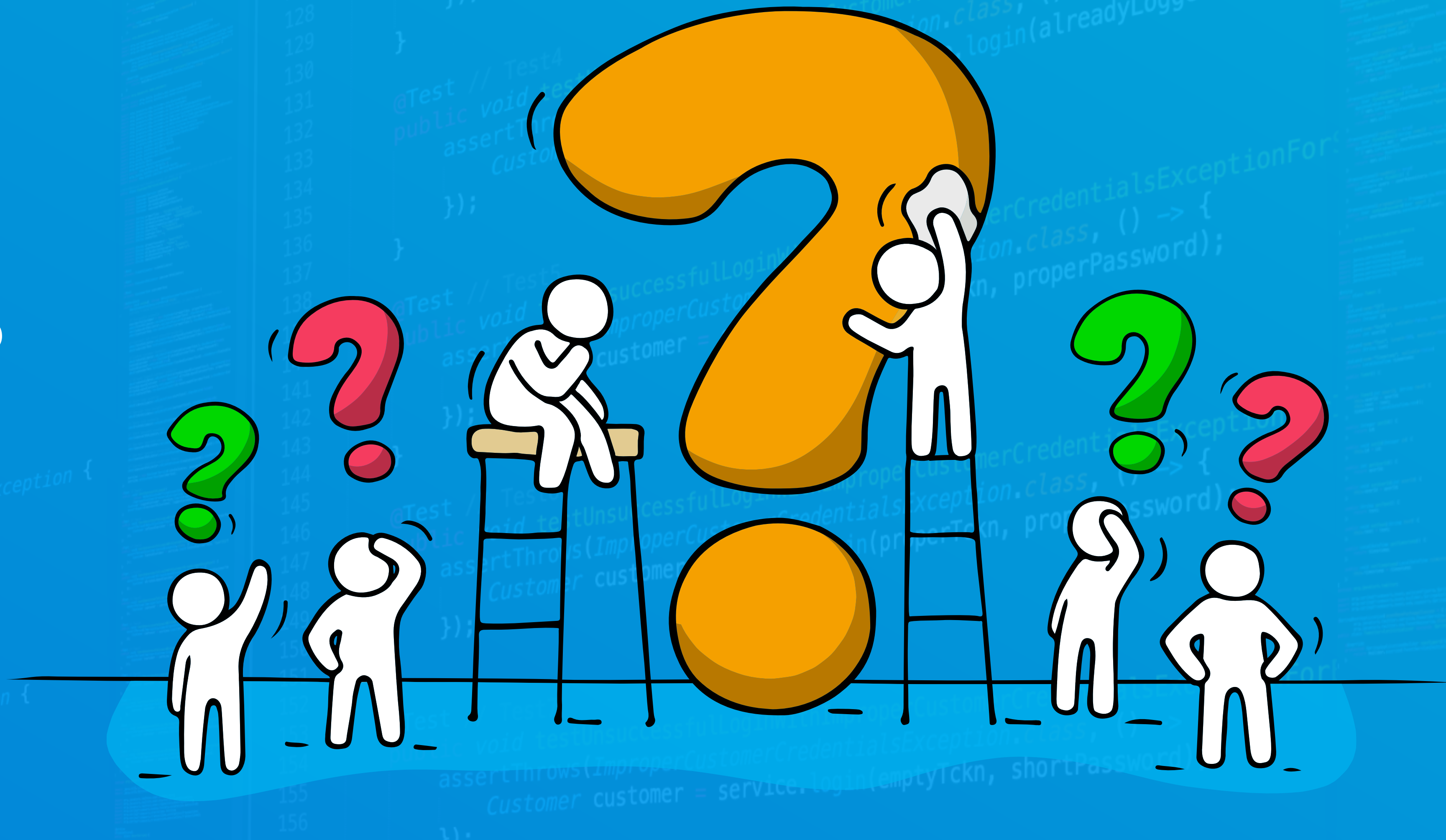
- This implementation is a good application of what is emphasized by GoF:
- *A class is more reusable when you minimize the assumptions other classes must make to use it.*
- Building the adaptation into a small, reusable adapter driven by a narrow interface: the *smallest* set of operations the client truly needs.
- Narrow interface means depending on the minimum: The less one adapter adapts, the more sources it can serve => Interface Segregation Principle

Parameterized Adapters - V



- The interface of different power systems are reduced to only one method, that is in the form of `void f()`.
- If this form doesn't fit into the power systems, different forms can be considered such as `R void f(T t)`, which can be represented by **Function** in Java.

Questions?





Consequences

Consequences - I



- The adapter pattern enables us to use an object that is useful but doesn't the interface we want or know how to use.
- It does this by wrapping the object in an adapter without modifying its original interface.
- The new interface of the incompatible object is now defined by the adapter object.
- The adapter has become the wrapper for the incompatible object.

Consequences - II



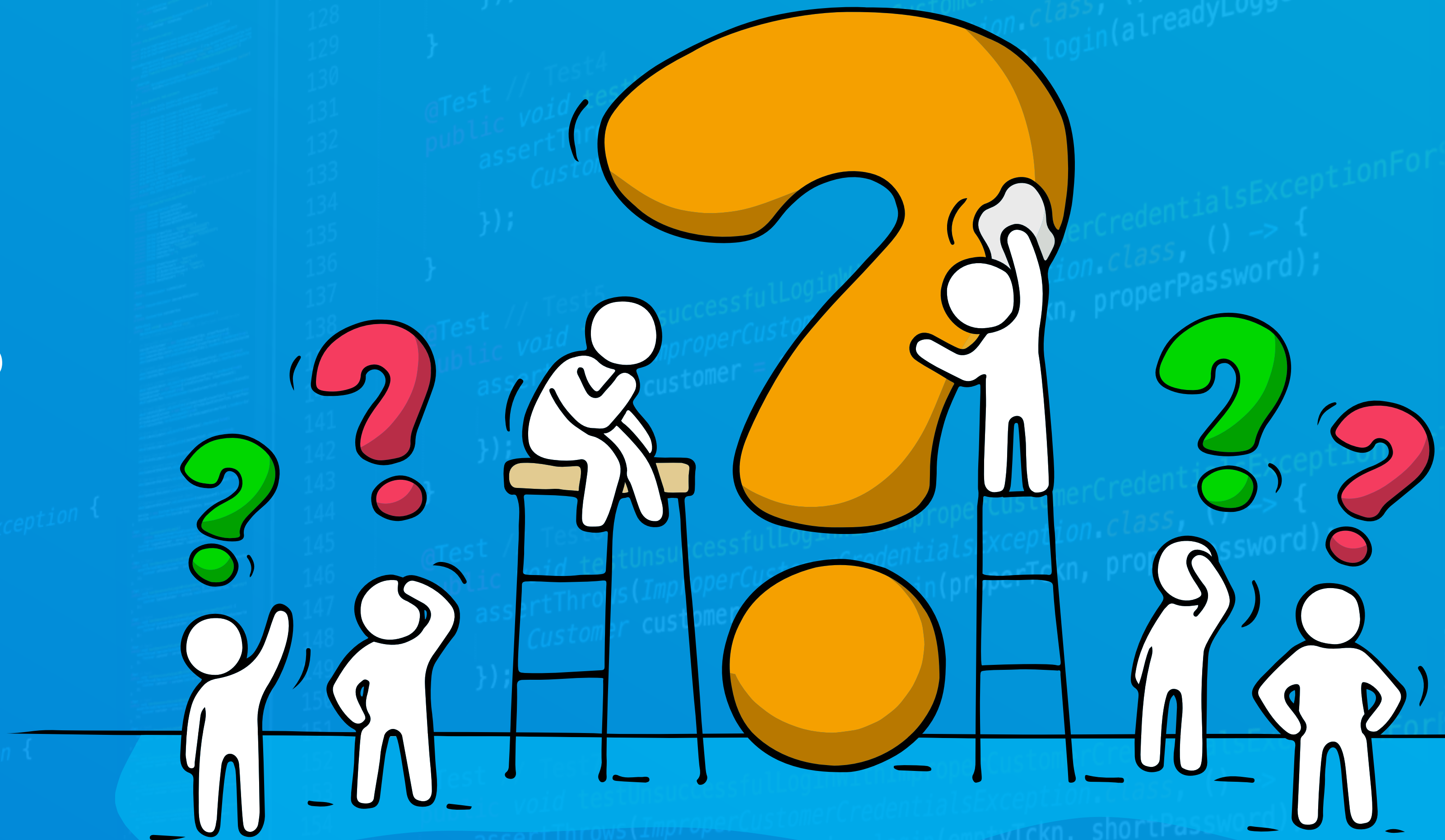
- Additionally, the adapter can be designed to be bidirectional.
- If the incompatible object needs to conform to multiple interfaces, a separate adapter can be created for each interface.

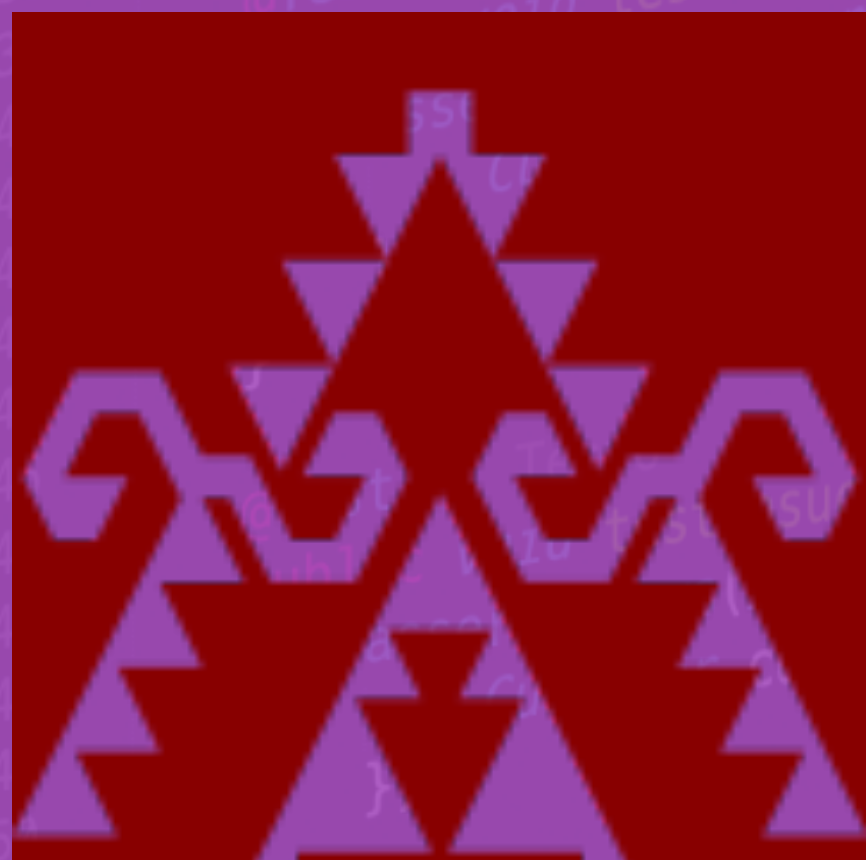
Consequences - III



- Additionally, the adapter can be designed to be bidirectional.
- If the incompatible object needs to conform to multiple interfaces, a separate adapter can be created for each interface.
- Or a pluggable adapter for all incompatible interfaces could also be developed using the mechanisms of functional programming
- This can be done by reducing the adapter's assumptions about adaptees.
- This makes the adapter much more reusable.

Questions?





Usages

Ports and Adapters - I



- Main example of the **Adapter** pattern is *Ports and Adapters* architectural style or pattern, which is also known as *Hexagonal Architecture*.
- <https://alistair.cockburn.us/hexagonal-architecture>
- Adapters are the interfaces that are exposed by the domain to communicate with all non-domain or external parts of the application such as databases, message queues, HTTP clients, transaction or security managing elements etc.
- Ports are implementation of adapters by external elements.

Ports and Adapters - II

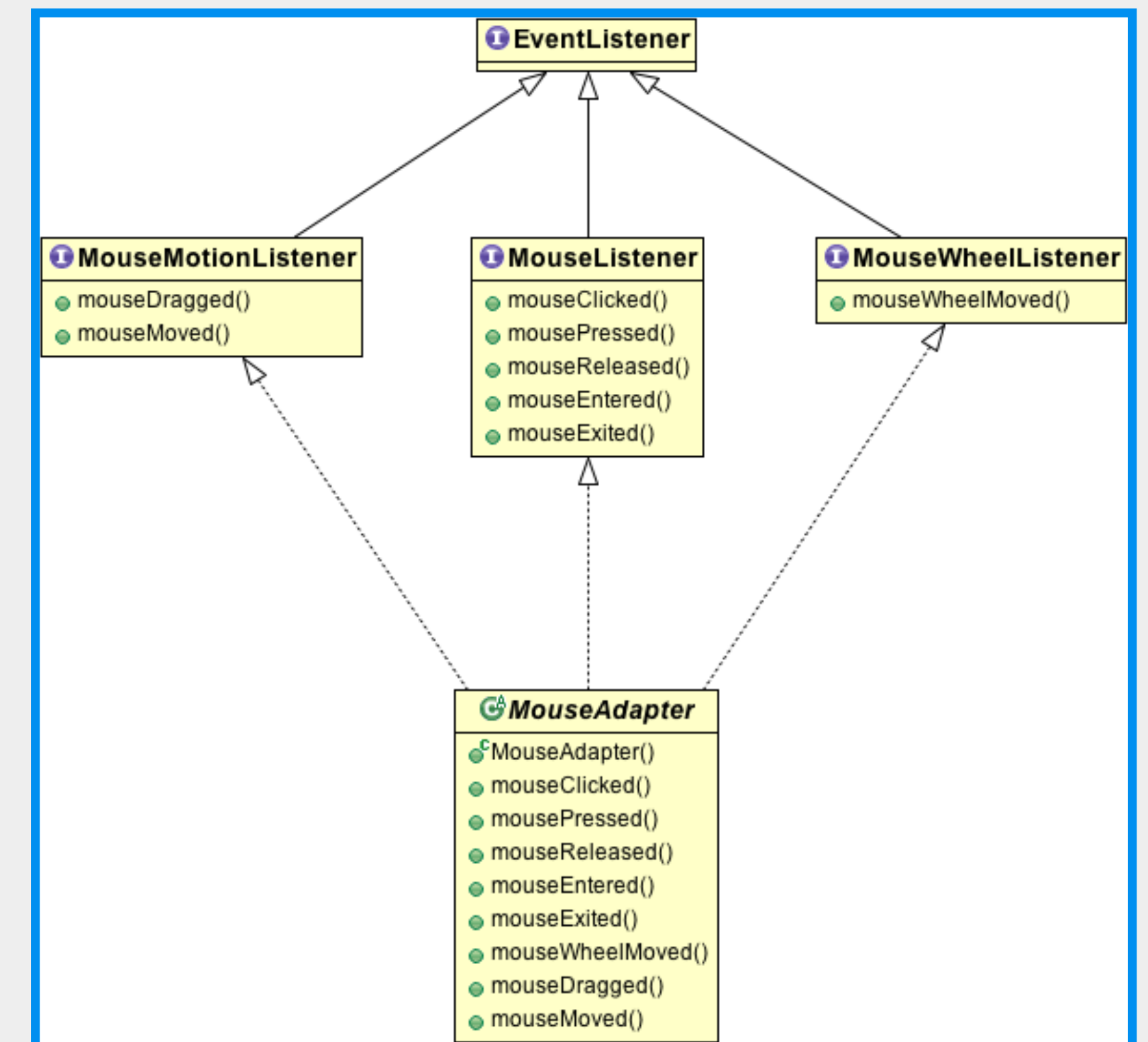


- *Ports and Adapters* architectural style or pattern is used to allow an application to equally be driven by users, programs, automated test or batch scripts, and to be developed and tested in isolation from its eventual run-time devices and databases.

Usages in Java - I



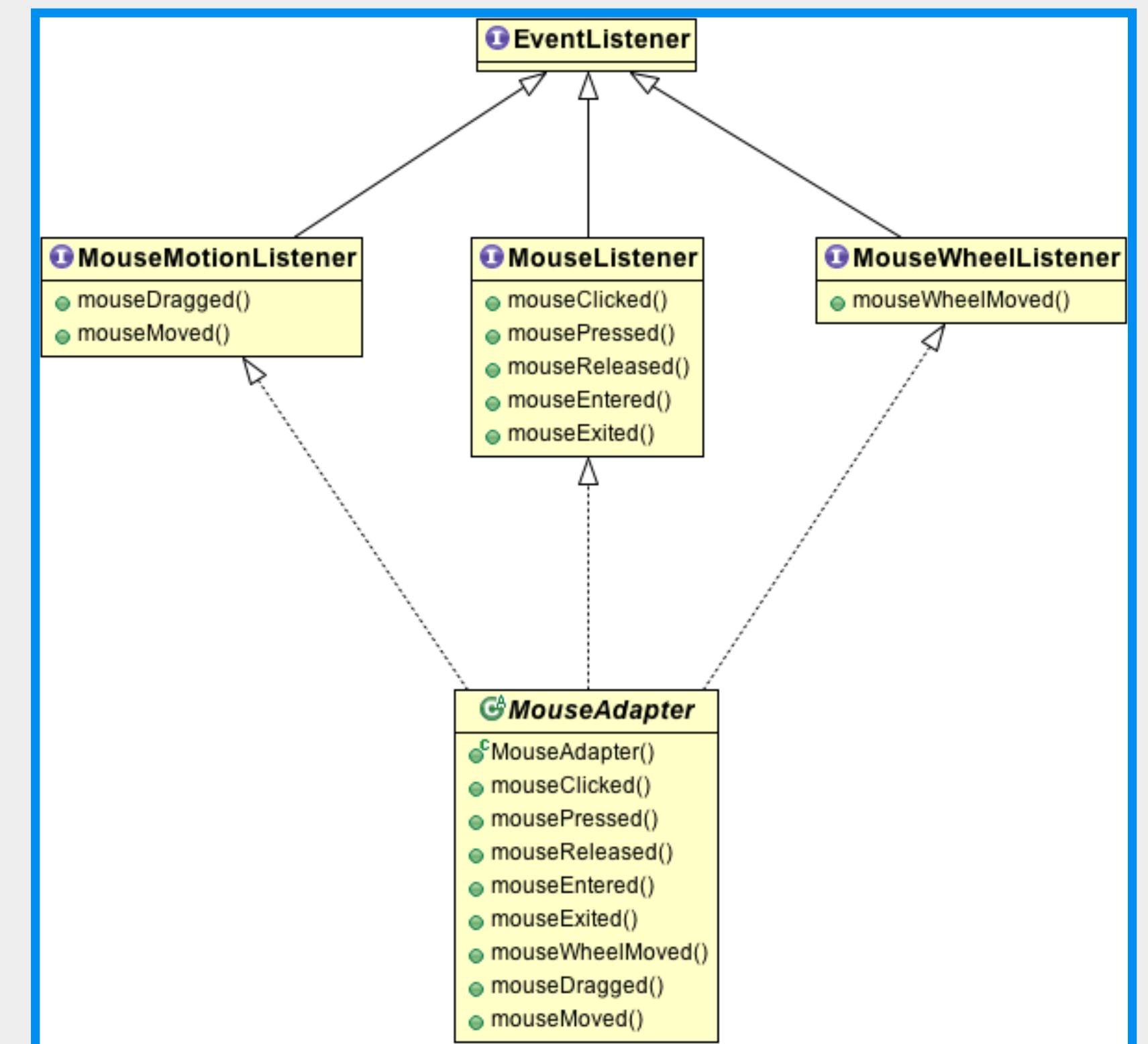
- The `java.awt.event` package contains many adapter classes for listeners in AWT, and the `javax.swing.event` package contains many adapter classes for listeners in Swing.
- In Java, adapters such as `java.awt.event.MouseAdapter` exist to eliminate the need to inherit multiple interfaces and override all AWT methods.



Usages in Java - II



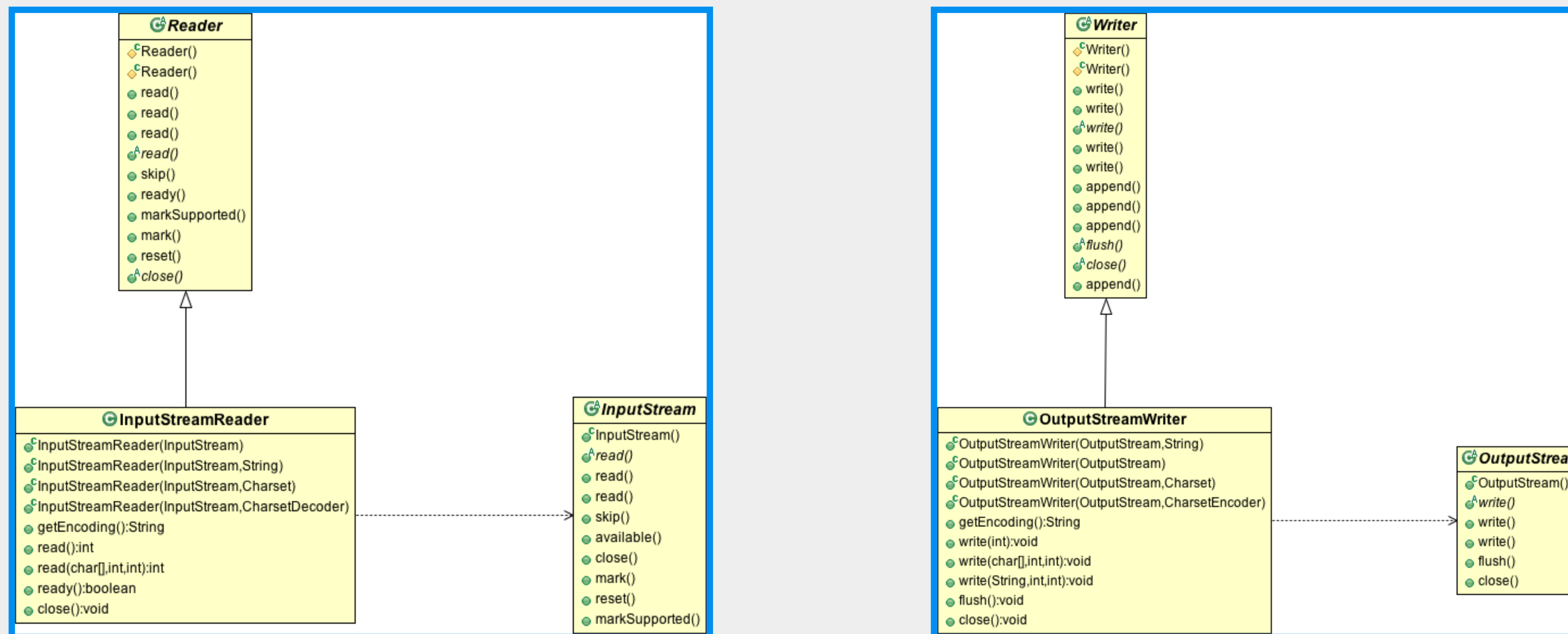
- These adapters make listener interfaces more usable and reduce the number of methods that need to be overridden.



Usages in Java - III



- Additionally, the `InputStreamReader` and `OutputStreamWriter` classes in the `java.io` package act as a bridge between byte and character streams.



Usages - IV



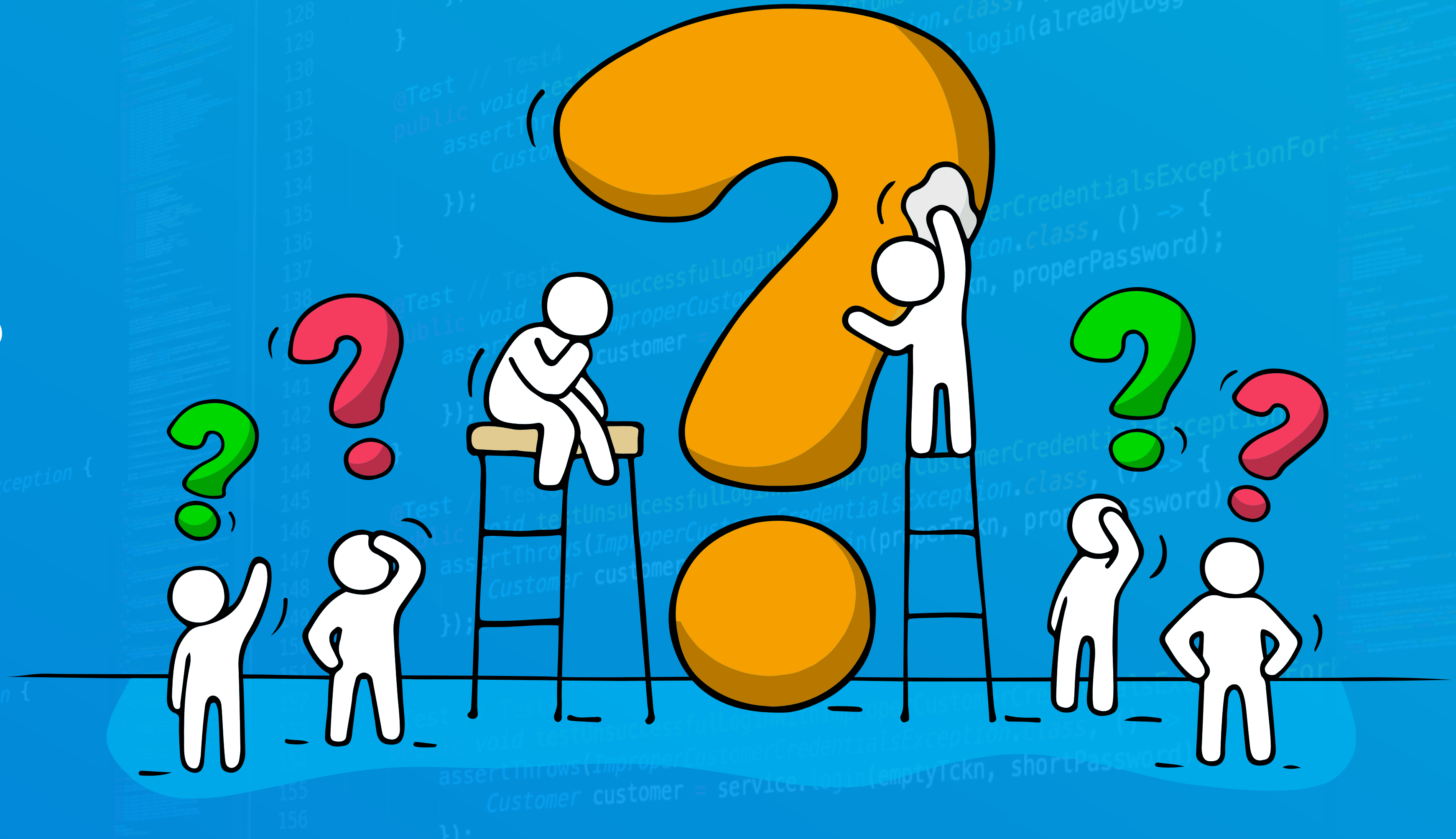
- In .NET, the `System.Web.UI.Adapters` namespace contains abstract classes for control and page adapters.
- For example, the `ControlAdapter` class is designed to eliminate differences in how different browsers render HTML.
- `ControlAdapter` can also perform filtering.
- `PageAdapter` is an abstract base class for all page adapters.

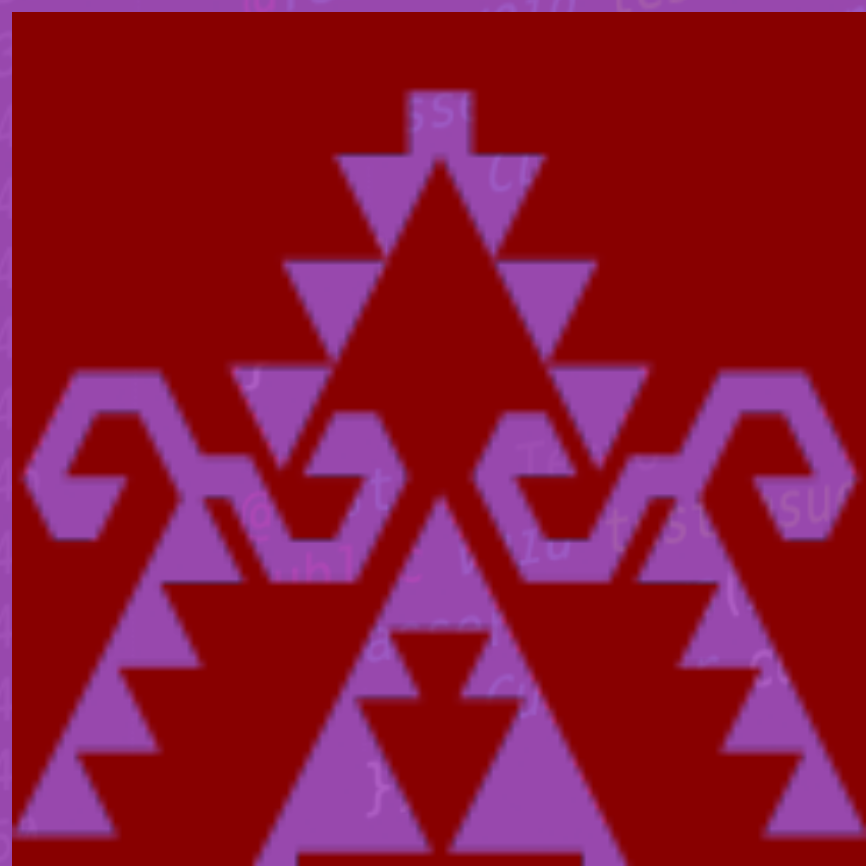
Usages - V



- The **Adapter** pattern is also frequently used in integration layers.
- XML is a markup language with its own types and structures.
- CORBA is a distributed computing framework that is independent of programming languages.
- In different programming languages, such as C#/.NET, adapters are used to resolve incompatibilities with XML and CORBA.

Questions?





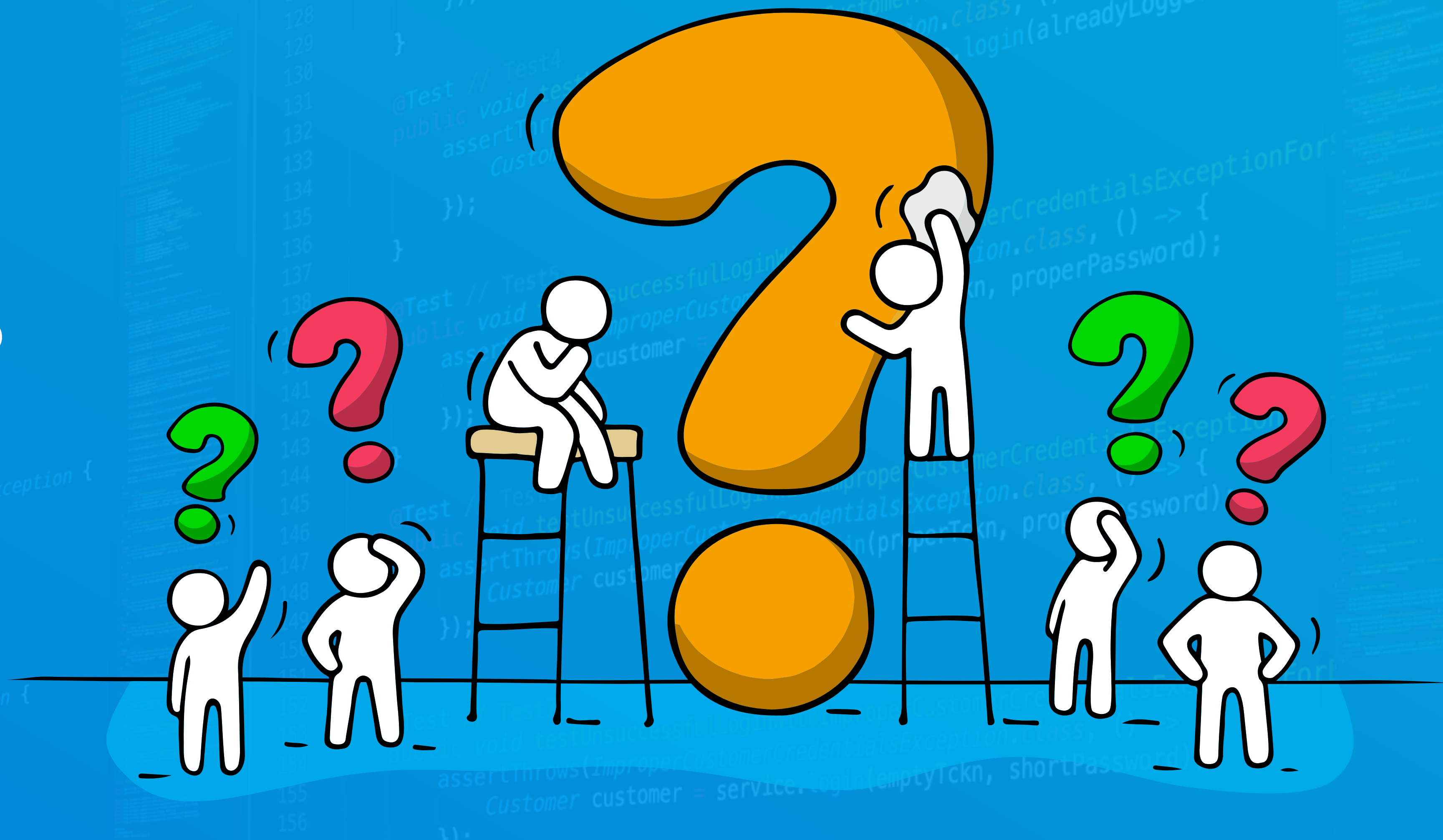
Relationship to Other Patterns

Relationships



- There are some patterns that are similar to **Adapter** such as **Bridge**, **Decorator** and **Proxy**
- **Bridge** has a totally different intent.
- **Decorator** enhances another object but doesn't change its interface.
- **Proxy** hides an object and doesn't change its interface.

Questions?



To Dos



Relationships



- Update Usages section using "2 - Adapter - Usages-slides.md".
- Homeworks

End of the Chapter

Questions?

